# 알고리즘 중간고사 총정리

---

## 목차


- 1장. 알고리즘이란 무엇인가
- 1.1 우리가 풀어야 할 질문
- 1.2 알고리즘의 정의
- 1.3 유클리드 GCD — 가장 오래된 알고리즘
- 1.4 고전 문제와 $\log_2 n$ 모티프
- 1.4.1 이진 탐색
- 1.4.2 독이 든 포도주와 이진 인코딩
- 1.4.3 가짜 동전 (1,024개)
- 1.5 오일러 회로와 그래프
- 1장 정리
- 2장. 복잡도 분석 — Big-O, 점화식, 마스터 정리
- 2.1 왜 복잡도를 분석하는가
- 2.1.1 RAM 모델
- 2.1.2 복잡도 분석의 네 가지 유형
- 2.2 점근적 표기법
- 2.2.1 Big-O — 점근적 상한
- 2.2.2 Big-Omega — 점근적 하한
- 2.2.3 Theta — 꼭 맞는 한계

- 2.2.4 복잡도 계층
- 2.3 점화식과 재귀 분석
- 2.3.1 방법 1 — 반복 대입
- 2.3.2 방법 2 — 추측 후 귀납 증명
- 2.3.3 방법 3 — 마스터 정리
- 2.4 알고리즘 설계 여섯 가지 기법
- 2장 정리
- 3장. 자료구조와 기본 정렬
- 3.1 핵심 자료구조
- 3.1.1 배열 vs 연결 리스트
- 3.1.2 스택 — LIFO
- 3.1.3 큐 — FIFO
- 3.1.4 힙 — 우선순위 큐
- 3.2 기본 정렬 — $\Theta(n^2)$
- 3.2.1 선택 정렬
- 3.2.2 버블 정렬
- 3.2.3 삽입 정렬
- 3.3 고급 정렬 — $\Theta(n \log n)$
- 3.3.1 병합 정렬
- 3.3.2 퀵 정렬
- 3.3.3 힙 정렬
- 3.3.4 세 정렬 비교표
- 3.4 비교 기반 정렬의 하한
- 3.5 선형 시간 정렬 — 비교 기반 하한 우회
- 3.5.1 계수 정렬
- 3.5.2 기수 정렬
- 3장 정리
- 4장. 분할 정복
- 4.1 분할 정복 패러다임
- 4.2 대표 알고리즘 카탈로그
- 4.3 선택 문제
- 4.3.1 랜덤 선택 — 기대 $\Theta(n)$
- 4.3.2 중앙값의 중앙값 — 최악 $\Theta(n)$
- 4.4 최근접 점 쌍
- 4.5 분할 정복이 실패하는 경우

- 4장 정리
- 5장. 그리디 알고리즘
- 5.1 그리디 패러다임
- 5.2 정확성을 위한 두 필수 조건
- 5.3 그리디 실패 사례
- 5.4 그리디 알고리즘 카탈로그
- 5.4.1 동전 거스름돈 (표준 화폐)
- 5.4.2 분할 가능 배낭
- 5.4.3 활동 선택
- 5.4.4 작업 스케줄링 (활동 선택과 혼동 주의)
- 5.4.5 허프만 코딩 — 최적 접두 코드
- 5.5 최소 신장 트리 (MST)
- 5.5.1 크루스칼 — 간선 중심
- 5.5.2 프림 — 정점 중심
- 5.5.3 크루스칼과 프림의 비교
- 5.6 다익스트라 — 최단 경로
- 5장 정리
- 6장. 동적 계획법
- 6.1 DP 의 본질
- 6.2 필수 두 조건
- 6.3 두 가지 구현 전략
- 6.3.1 메모이제이션 — 하향식
- 6.3.2 테이블화 — 상향식
- 6.3.3 비교
- 6.4 DP 6단계 레시피
- 6.5 대표 DP 문제 카탈로그
- 6.5.1 피보나치 — 예열
- 6.5.2 최장 공통 부분 수열 (LCS)
- 6.5.3 편집 거리 (Edit Distance, Levenshtein)
- 6.5.4 0/1 배낭
- 6.5.5 동전 거스름돈 (최소 동전 개수)
- 6.5.6 플로이드-워셜 — 모든 쌍 최단 경로
- 6.5.7 행렬 경로 (최대 합)
- 6.6 DP 문제 총정리
- 6장 정리

- 7장. 중간고사 복습 — 종합
- 7.1 학기 전체를 관통하는 흐름
- 7.2 복잡도 치트시트
- 7.2.1 복잡도 계층
- 7.2.2 정렬 완전 표
- 7.2.3 마스터 정리 빠른 참조
- 7.2.4 점화식 카탈로그
- 7.3 DP 점화식 모음
- 7.4 자주 하는 실수와 예방
- 8장. 큰 그림 — 세 패러다임의 관계
- 8.1 세 패러다임 비교
- 8.2 결정 흐름도
- 8.3 같은 문제, 다른 패러다임
- 8.4 핵심 연결 고리 일곱 가지
- 연결 1: 재귀적 사고의 통일성
- 연결 2: 분할 정복과 로그 복잡도
- 연결 3: 그리디에서 DP 로의 전환
- 연결 4: 비교 하한과 선형 정렬의 위계
- 연결 5: Prim ↔ Dijkstra ↔ Huffman
- 연결 6: 분할 정복의 한계에서 DP 로
- 연결 7: 복잡도 분석이 모든 것을 연결
- 8.5 답안 작성 심화 팁
- 패턴 1 — "왜" 를 먼저
- 패턴 2 — 트레이드오프
- 패턴 3 — 개념 연결
- 패턴 4 — 한계 언급
- 패턴 5 — 실무 예시
- 9장. 시험 직전 체크리스트
- 9.0 주차별 핵심 치트시트
- 다섯 가지 시험 팁
- 9.1 자주 하는 실수
- 9.2 시험 문제 유형 예시
- 예시 1 — 복잡도 분석 (마스터 정리 적용)
- 예시 2 — 그리디 vs DP (동전 거스름돈)
- 예시 3 — DP

- 9.3 패러다임 식별 연습 10문제
- 문제 1-5
- 문제 6-10
- 패러다임 결정 플로우차트
- 9.4 복습 문제 다섯 가지
- 복습 1 — 복잡도 분석
- 복습 2 — 병합 정렬
- 복습 3 — 분할 정복으로 최댓값
- 복습 4 — 활동 선택
- 복습 5 — 계단 오르기 DP
- 9.6 자가 점검 30문제
- 1주차 기초
- 2주차 복잡도
- 3주차 자료구조·정렬
- 4주차 분할 정복
- 5주차 그리디
- 6주차 DP
- 9.7 계산 연습
- 마스터 정리
- LCS
- 0/1 배낭
- Big-O 증명
- 9.8 의사코드 체크
- 이진 탐색
- 병합 정렬
- LCS 채우기
- 0/1 배낭 채우기
- Dijkstra
- 9.9 시험 당일 전략
- 시간 배분 (1시간 기준)
- 답안 작성 6요소 템플릿
- 체크 포인트 — 자주 놓치는 것
- 맺음말


---

# 1장. 알고리즘이란 무엇인가

## 1.1 우리가 풀어야 할 질문

문제를 푸는 방법은 수없이 많다. 같은 문제에 대한 두 가지 방법이 있을 때 **무엇이 더 좋은가, 왜 더 좋은가** 를 객관적으로 판단해야 한다. 이를 위해서는 다음 세 가지가 필요하다.

1. "알고리즘이 무엇인가" 에 대한 정확한 정의가 필요하다.
2. "효율적이다" 가 무엇을 뜻하는지 측정 가능한 척도가 필요하다.
3. 반복적으로 등장하는 설계 패턴을 익혀야 한다.

이 세 가지가 1주차의 핵심이며, 이후 주차들은 이 기반 위에 더 복잡한 알고리즘을 쌓아 올린다.

## 1.2 알고리즘의 정의

**알고리즘(Algorithm)** 은 입력을 받아 유한한 단계를 거쳐 원하는 출력을 생성하는 체계적인 풀이 절차이다.

```
┌──────────┐      ┌──────────────┐      ┌──────────┐
│ 입력     │ ──→  │ 알고리즘     │ ──→  │ 출력     │
│ (문제)   │      │ (유한  단계) │      │ (결과)   │
└──────────┘      └──────────────┘      └──────────┘
```

반드시 만족해야 할 다섯 가지 속성은 다음과 같다.

| 속성 | 의미 | 실패 예 |
|---|---|---|
| 정확성 (Correctness) | 모든 유효 입력에 대해 올바른 출력을 낸다. | 잘못된 답을 반환한다. |
| 실행 가능성 (Executability) | 각 단계가 기계적으로 실행 가능하다. | "오라클에 물어본다" 처럼 실행 불가능한 단계를 둔다. |
| 유한성 (Finiteness) | 유한한 시간 안에 반드시 종료한다. | 무한 루프가 발생한다. |
| 효율성 (Efficiency) | 시간과 공간을 적절히 사용한다. | 지수 시간처럼 실행 불가능한 자원을 소모한다. |
| 명확성 (Unambiguity) | 모호하지 않게 기술되어 있다. | "상황에 맞게 처리한다" 같은 모호한 서술을 쓴다. |

시험에서 "이 설명이 알고리즘인가?" 를 묻는 문제가 나오면 이 다섯 가지 속성을 기준으로 판단한다.

니클라우스 비르트(Niklaus Wirth)의 법칙 "Algorithm + Data Structure = Program" 은 이 관계를 요약한다. 알고리즘이 "무엇을 할지" 를 결정한다면 자료구조는 "어떻게 저장할지" 를 결정하며, 둘은 분리할 수 없다. 정렬되지 않은 사전에서는 순차 탐색만 가능하지만, 정렬된 사전에서는 이진 탐색이 가능하다는 점이 그 예시이다.

## 1.3 유클리드 GCD — 가장 오래된 알고리즘

**배경** — 유클리드 GCD 는 기원전 약 300년에 고안되어, 컴퓨터가 등장하기 2,300년 전부터 알려진 알고리즘이다.

**핵심 항등식:**

$$\gcd(a, b) = \gcd(b, a \bmod b), \qquad \gcd(a, 0) = a$$

**왜 성립하는가** — $a = q \cdot b + r$ ($r = a \bmod b$) 이라고 하면, $(a, b)$ 의 공약수 집합과 $(b, r)$ 의 공약수 집합은 동일하다. 따라서 두 쌍의 최대공약수도 같다.

**구현:**

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

**설명** — 매 반복마다 $(a, b) \to (b, a \bmod b)$ 로 갱신. 항등식 $\gcd(a, b) = \gcd(b, a \bmod b)$ 에 의해 GCD 값이 보존된다. $b$ 가 매 단계 엄격히 감소하므로 반드시 $b = 0$ 에 도달하고 그때 $a$ 가 최종 GCD. 튜플 동시 대입(`a, b = b, a % b`)으로 변수 순서를 교환하는 파이썬 관용구.

**gcd(48, 18) 추적:**

| 단계 | a | b | a mod b |
|---|---|---|---|
| 1 | 48 | 18 | 12 |
| 2 | 18 | 12 | 6 |
| 3 | 12 | 6 | 0 |
| 4 | 6 | 0 | → 반환 6 |

**복잡도** — 최악 $O(\log \min(a, b))$. 최악은 연속한 피보나치 수 쌍에서 발생.

재귀(Recursion) 구현도 가능하며, "더 작은 문제" 로 환원하는 **재귀적 사고** 의 원형. 이 사고가 4주차 분할 정복(Divide and Conquer)으로 이어진다.

## 1.4 고전 문제와 $\log_2 n$ 모티프

1주차 의 고전 문제들은 모두 **"문제를 절반으로 줄이면 $\log n$ 성능"** 이라는 동일한 아이디어를 다른 각도에서 보여준다.

| 문제 | 전략 | 최악 | 핵심 아이디어 |
|---|---|---|---|
| 최댓값 찾기 | 순차 스캔 | $O(n)$ | 모든 원소를 한 번은 봐야 함 |
| 이진 탐색(Binary Search) | 분할 정복 | $O(\log n)$ | 정렬(Sort / Sorting)된 배열(Array)에서 중간값과 비교해 절반 제거 |
| 동전 거스름돈(Coin Change) (표준) | 그리디 | $O(k)$ | 큰 화폐부터 선택 |
| 오일러 회로(Euler Circuit) | 그래프(Graph) 순회 | $O(V+E)$ | 모든 정점(Vertex) 차수(Degree)가 짝수일 때 존재 |
| 미로 (벽 연결) | 오른손 법칙 | $O(\text{크기})$ | DFS 의 물리적 버전 |
| 가짜 동전(Fake Coin) (1,024 개) | 반으로 나누기 | **10 회 측정** | $\log_2 1024 = 10$ |
| 독이 든 포도주 (n 항아리) | 이진 인코딩(Binary Encoding) | $\lceil \log_2 n \rceil$ 명 | 각 시종 = 1 비트 정보 |

### 1.4.1 이진 탐색

**전제** — 배열이 **정렬되어 있어야** 함.

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:                    # [주의] 등호 필수!
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

**설명** — `[left, right]` 가 "아직 확인하지 않은 폐구간" 이라는 불변식을 유지. 매 반복에서 중간값 `arr[mid]` 와 `target` 을 비교해 폐구간을 **절반으로 축소**. 같으면 인덱스 반환, 타겟이 더 크면 왼쪽 절반을 버리고( `left = mid + 1` ), 작으면 오른쪽 절반을 버린다( `right = mid - 1` ). `left > right` 가 되면 남은 구간이 공집합 → -1 반환. 복잡도 $\Theta(\log n)$.

**`left <= right` 의 등호가 빠지면** 구간에 원소 하나만 남은 경우( `left == right` )를 검사하지 않아 존재하는 키를 놓친다. 등호는 "단일 원소 확인" 단계를 포함시키는 역할.

**비교** — 원소가 100만 개일 때 순차 탐색은 최악 1,000,000 회 비교가 필요하지만 이진 탐색은 약 20 회($\log_2 10^6 \approx 20$)만 필요하다.

### 1.4.2 독이 든 포도주와 이진 인코딩

**문제** — $n$ 개의 항아리 중 하나에 독이 들어 있다. 독은 일주일 뒤에 효과가 나타난다. 일주일 안에 독이 든 항아리를 반드시 찾아내되 시음할 시종의 수를 최소화해야 한다.

**전략** — 각 항아리에 $0$ 부터 $n-1$ 까지의 이진 ID 를 부여한다. 시종 $i$ 는 자신의 비트 $i$ 가 $1$ 인 모든 항아리를 마신다.

**4개 항아리 예시:**

| 항아리 | 이진 | 시종 A (비트 1) | 시종 B (비트 0) |
|---|---|---|---|
| 0 | 00 | — | — |
| 1 | 01 | — | ✓ |
| 2 | 10 | ✓ | — |
| 3 | 11 | ✓ | ✓ |

**결과 해석** — 예를 들어 A 가 사망하고 B 가 생존했다면 이진수 10 에 해당하는 2번 항아리가 독이다.

**일반화** — $n$ 개의 항아리에는 $\lceil \log_2 n \rceil$ 명의 시종이 필요하며, 각 시종이 1비트의 정보를 제공한다. 정보 이론의 핵심 명제 "$n$ 가지 경우를 구별하려면 최소 $\log_2 n$ 비트가 필요하다" 가 이진 탐색이 $\log n$ 인 이유와 동일한 원리이다.

### 1.4.3 가짜 동전 (1,024개)

**문제** — 양팔 저울로 1,024개 동전 중 한 개의 가짜(조금 가벼움)를 찾되 측정 횟수를 최소화해야 한다.

| 접근 | 설명 | 최악 횟수 |
|---|---|---|
| 하나씩 비교 | 무차별 대입 | 1,023회 |
| 쌍으로 비교 | 약간 개선된 방식 | 512회 |
| 반으로 나누기 | 분할 정복 | 10회 |

**분할 정복 풀이** — 512 vs 512 에서 시작하여 가벼운 쪽을 선택한 뒤, 256 vs 256, 128 vs 128 과 같이 절반씩 줄여 나간다. $\log_2 1024 = 10$ 이므로 정확히 10회면 충분하다. 이 아이디어가 4주차의 병합 정렬과 이진 탐색의 근간이 된다.

## 1.5 오일러 회로와 그래프

**오일러 경로(Euler Path)** 는 모든 간선을 정확히 한 번씩 지나는 경로이고, **오일러 회로(Euler Circuit)** 는 오일러 경로이면서 시작점으로 돌아오는 경로이다.

**정리**:

- 오일러 **회로** 가 존재하려면 모든 정점의 차수가 짝수여야 한다.
- 오일러 **경로** (회로는 아님)가 존재하려면 홀수 차수 정점이 정확히 0개 또는 2개여야 한다.

**직관** — 회로에서는 한 정점에 들어왔으면 반드시 나가야 하므로, 각 정점의 들어오는 간선 수와 나가는 간선 수가 같아야 하고 따라서 차수가 짝수이다.

**다리(Bridge) 회피** — 다리는 제거 시 그래프가 끊어지는 간선이다. 다리를 너무 일찍 건너면 반대쪽에 갇혀 나머지 간선을 모두 순회할 수 없게 된다.

---

## 1장 정리

1주차의 큰 그림은 다음과 같다. 알고리즘은 정확성·실행 가능성·유한성·효율성·명확성의 다섯 가지 속성을 만족하는 체계적 절차이다. 유클리드 GCD 는 "더 작은 문제로 환원" 하는 재귀적 사고의 원형이다. 이진 탐색, 가짜 동전, 독이 든 포도주 문제에서 공통적으로 나타나는 $\log_2 n$ 모티프는 "절반 분할" 이 효율의 핵심임을 보여주며, 이 재귀적 분할 사고가 4주차의 분할 정복으로 확장된다.

---

# 2장. 복잡도 분석 — Big-O, 점화식, 마스터 정리

## 2.1 왜 복잡도를 분석하는가

같은 문제를 푸는 두 알고리즘이 있을 때 "어느 쪽이 더 빠른가" 를 하드웨어와 무관하게 비교할 방법이 필요하다. 실행 시간(초)은 하드웨어에 의존하지만 기본 연산 횟수는 알고리즘 고유의 성질이므로, 연산 횟수를 입력 크기 $n$ 의 함수로 표현한다.

충분히 큰 $n$ 에서는 노트북에서 돌리는 $O(n\log n)$ 알고리즘이 슈퍼컴퓨터에서 돌리는 $O(n^2)$ 알고리즘보다 빠르다. 하드웨어는 상수 배율만 줄이지만 알고리즘은 복잡도의 차수 자체를 바꾸기 때문이다.

### 2.1.1 RAM 모델

**RAM (Random Access Machine)** 은 이론적 분석의 표준 계산 모델이다. 기본 연산(대입·비교·산술·배열 접근)은 각 1시간 단위로 간주하며, 메모리 접근 시간은 주소와 무관하게 일정하다고 가정한다. 기본 연산에는 사칙연산, 비교, 대입, 배열 인덱싱, 포인터 역참조 등이 포함된다.

### 2.1.2 복잡도 분석의 네 가지 유형

| 유형 | 의미 | 주로 쓰이는 상황 |
|---|---|---|
| 최악의 경우 (Worst-case) | 어떤 입력이 들어와도 이 이상 걸리지 않는다는 상한을 보장한다. | 가장 일반적으로 쓰인다. |
| 평균의 경우 (Average-case) | 확률 분포상 기대 시간을 나타낸다. | 실용적이지만 계산이 복잡하다. |
| 최선의 경우 (Best-case) | 가장 빠르게 실행되는 경우이다. | 알고리즘 비교 기준으로는 쓰이지 않는다. |
| 분할 상환 (Amortized) | 일련의 연산에 대한 평균을 분석한다. | 동적 배열의 확장 연산 등에 쓰인다. |

최악의 경우를 주로 쓰는 이유는 "보장" 때문이다. "이 알고리즘은 어떤 입력에 대해서도 $n^2$ 을 넘지 않는다" 는 약속이 바로 최악의 경우 분석이다.

## 2.2 점근적 표기법

정확한 연산 횟수(예: $T(n) = 3n^2 + 100n + 5000$)는 구현 세부에 의존하지만, $n$ 이 충분히 크면 최고 차항만 중요해진다. 이 관찰을 엄밀하게 표현하는 도구가 점근적 표기법이다.

### 2.2.1 Big-O — 점근적 상한

**정의**:

$$O(g(n)) = \ f(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } \forall n \geq n_0, f(n) \leq c \cdot g(n)$$

직관적으로 "$f$ 가 $g$ 보다 더 빠르게 자라지는 않는다" 를 뜻하며, 상수 배율은 무시한다.

**다항식 증명 레시피** — $f(n) = a_k n^k + \cdots + a_0$ 에 대해 $c = |a_k| + |a_{k-1}| + \cdots + |a_0|$ 와 $n_0 = 1$ 로 잡으면 모든 $n \geq 1$ 에서 $f(n) \leq c \cdot n^k$ 가 성립한다.

**예시** — $2n^2 - 8n + 3 = O(n^2)$ 은 $c = 13$, $n_0 = 1$ 로 증명된다. 실제로 $|2n^2 - 8n + 3| \leq 2n^2 + 8n + 3 \leq 13n^2$ 이 모든 $n \geq 1$ 에서 성립한다.

### 2.2.2 Big-Omega — 점근적 하한

$$\Omega(g(n)) = \ f(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } \forall n \geq n_0, f(n) \geq c \cdot g(n)$$

직관적으로 "$f$ 가 $g$ 보다 더 느리게 자라지는 않는다" 를 뜻한다.

### 2.2.3 Theta — 꼭 맞는 한계

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$

"$f$ 와 $g$ 가 (상수 배율을 제외하면) 같은 속도로 자란다" 를 뜻한다.

**세 표기법의 구분**:

- $O(n^2)$ 은 "최대 $n^2$ 수준" 을 의미한다.
- $\Omega(n^2)$ 은 "최소 $n^2$ 수준" 을 의미한다.
- $\Theta(n^2)$ 은 "정확히 $n^2$ 수준" 을 의미한다.

**타이트하게 쓰기** — $n\log n = O(n^2)$ 은 참이지만 느슨해서 무용지물이다. 대신 $O(n\log n)$ 또는 $\Theta(n\log n)$ 으로 써야 정보력이 있다.

### 2.2.4 복잡도 계층

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n\log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

**입력 크기별 증가 (실감 표)**:

| n | $\log n$ | $n$ | $n\log n$ | $n^2$ | $2^n$ |
|---|---|---|---|---|---|
| 10 | 3.3 | 10 | 33 | 100 | 1,024 |
| 100 | 6.6 | 100 | 664 | $10^4$ | $10^{30}$ |
| $10^6$ | 20 | $10^6$ | $2 \cdot 10^7$ | $10^{12}$ | — |

**10억 개 원소 정렬 비교** (초당 $10^8$ 연산인 PC 기준):

- $O(n^2)$ 알고리즘으로는 약 300년이 걸려 실행이 불가능하다.
- $O(n\log n)$ 알고리즘으로는 약 5분이면 끝난다.

이것이 3주차에서 $O(n^2)$ 정렬(선택·버블·삽입)을 버리고 $O(n\log n)$ 정렬(병합·퀵·힙)을 쓰는 이유이다.

## 2.3 점화식과 재귀 분석

**점화식(Recurrence Relation)** 은 함수의 값을 더 작은 입력값으로 표현한 식이다.

**예시** — 다음은 대표적인 점화식이다.

- $T(n) = T(n-1) + c$ 는 팩토리얼 형태이다.
- $T(n) = 2T(n/2) + n$ 은 병합 정렬의 점화식이다.
- $T(n) = T(n/2) + 1$ 은 이진 탐색의 점화식이다.

"점화식을 푼다" 는 것은 이를 닫힌 형태(closed form)로 표현하는 작업을 뜻한다. 예를 들어 $T(n) = \Theta(n\log n)$ 으로 정리하는 것이다.

### 2.3.1 방법 1 — 반복 대입

점화식을 단계별로 전개하여 패턴을 찾은 뒤 일반화.

$T(n) = 2T(n/2) + n$, $T(1) = 1$, $n = 2^k$ 가정:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2(2T(n/4) + n/2) + n = 4T(n/4) + 2n \\ &= 8T(n/8) + 3n \\ &\vdots \\ &= 2^k T(n/2^k) + kn \end{aligned}$$

$2^k = n$ 이므로 $k = \log_2 n$:

$$T(n) = n \cdot T(1) + n\log_2 n = \Theta(n\log n)$$

### 2.3.2 방법 2 — 추측 후 귀납 증명

$T(n) = 2T(n/2) + n$ 에 대해 $T(n) = O(n\log n)$ 증명:

**기저** — $T(2) = 4$. $c \geq 2$ 면 $4 \leq 2c \cdot \log 2 = 2c$ 성립.

**귀납 가정** — $T(n/2) \leq c(n/2)\log(n/2)$.

**귀납 단계(Inductive Step)**: $$ \begin{aligned} T(n) &= 2T(n/2) + n \ &\leq 2 \cdot c(n/2) \log(n/2) + n \ &= cn(\log n - \log 2) + n \ &= cn \log n - cn + n \ &\leq cn \log n \quad (c \geq 1) \end{aligned} $$

**자주 하는 실수** — 귀납 단계에서 $cn\log n + 1$ 같은 여분 항이 남았을 때 "Big-O 니까 괜찮다" 고 넘기면 안 된다. 대신 추측을 $cn\log n - d$ 로 **강화** 하여 여분 항을 흡수할 여유를 만들어야 한다.

### 2.3.3 방법 3 — 마스터 정리

**형태**: $T(n) = a \cdot T(n/b) + f(n)$ $(a \geq 1, b > 1)$

**비교 함수**: $h(n) = n^{\log_b a}$ (재귀 트리(Recursion Tree)의 리프 비용)

**3 가지 경우**:

| Case | 조건 | 결과 | 의미 |
|---|---|---|---|
| 1 | $f(n) = O(n^{\log_b a - \varepsilon})$, 어떤 $\varepsilon > 0$ | $T(n) = \Theta(n^{\log_b a})$ | 리프 비용이 지배 |
| 2 | $f(n) = \Theta(n^{\log_b a})$ | $T(n) = \Theta(n^{\log_b a}\log n)$ | 각 레벨 비용 균형 |
| 3 | $f(n) = \Omega(n^{\log_b a + \varepsilon})$ + 정규성 조건 (Regularity Condition) | $T(n) = \Theta(f(n))$ | 루트 비용 지배 |

**정규성 조건 (Case 3)** — 어떤 $c < 1$ 에 대해 $af(n/b) \leq cf(n)$ 이 성립해야 한다. 이는 재귀 트리의 각 레벨 비용이 기하급수적으로 감소함을 의미한다.

**핵심 예제 카탈로그**:

| 점화식 | $a, b$ | $n^{\log_b a}$ | $f(n)$ | Case | 결과 |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| $T(n) = 2T(n/2) + n$ | 2, 2 | $n$ | $n$ | 2 | $\Theta(n\log n)$ |
| $T(n) = T(n/2) + 1$ | 1, 2 | 1 | 1 | 2 | $\Theta(\log n)$ |
| $T(n) = 2T(n/2) + 1$ | 2, 2 | $n$ | 1 | 1 | $\Theta(n)$ |
| $T(n) = 4T(n/2) + n$ | 4, 2 | $n^2$ | $n$ | 1 | $\Theta(n^2)$ |
| $T(n) = 4T(n/2) + n^2$ | 4, 2 | $n^2$ | $n^2$ | 2 | $\Theta(n^2\log n)$ |
| $T(n) = 3T(n/4) + n^2$ | 3, 4 | $n^{0.79}$ | $n^2$ | 3 | $\Theta(n^2)$ |

**마스터 정리를 쓸 수 없는 경우:**

- $T(n) = T(n-1) + \Theta(n)$ 은 반복 대입으로 $\Theta(n^2)$ 이 된다.
- $T(n) = T(n-1) + T(n-2) + 1$ 은 특성 다항식을 풀면 $\Theta(\phi^n)$ 이 된다.
- $T(n) = T(n/5) + T(7n/10) + n$ 은 귀납법과 기하급수 합으로 $\Theta(n)$ 이 된다.

마스터 정리는 4주차 분할 정복 알고리즘 분석의 핵심 도구이다.

## 2.4 알고리즘 설계 여섯 가지 기법

| 기법 | 영문 | 핵심 아이디어 | 대표 예시 |
|---|---|---|---|
| 분할 정복 | Divide and Conquer | 나누고 정복하고 결합한다. | 병합 정렬, 퀵 정렬, 이진 탐색 |
| 탐욕 알고리즘 | Greedy | 매 단계 지역 최적을 선택한다. | 다익스트라, 허프만, MST |
| 동적 계획법 | Dynamic Programming | 부분 문제를 저장하여 재사용한다. | LCS, 배낭, 편집 거리 |
| 근사 알고리즘 | Approximation | NP-난해 문제의 근사해를 구한다. | TSP 근사 |
| 백트래킹 | Backtracking | 완전 탐색에 가지치기를 더한다. | N-Queen |
| 분기 한정 | Branch and Bound | 백트래킹에 경계(bound)를 더해 탐색을 줄인다. | TSP 정확해 |

이 중 이 강의가 집중적으로 다루는 세 가지 핵심 기법은 분할 정복(4주차), 그리디(5주차), 동적 계획법(6주차)이다.

---

## 2장 정리

2주차의 큰 그림은 다음과 같다. 하드웨어와 무관한 비교를 위해 연산 횟수를 입력 $n$ 의 함수로 표현하며, Big-O 는 상한, $\Omega$ 는 하한, $\Theta$ 는 꼭 맞는 한계이다. 점화식은 재귀 알고리즘의 분석 도구이고, 마스터 정리는 분할 정복 점화식의 표준 해결 도구이다. 알고리즘 설계는 분할 정복·그리디·DP 의 세 핵심 패러다임으로 수렴한다.

---

# 3장. 자료구조와 기본 정렬

## 3.1 핵심 자료구조

### 3.1.1 배열 vs 연결 리스트

| 연산 | 배열 | 연결 리스트(Linked List) |
|---|---|---|
| **인덱스 접근** | **O(1)** | O(n) |
| **중간 삽입/삭제** | O(n) | **O(1)** (위치 알 경우) |
| **메모리** | 연속 (캐시 친화) | 흩어짐 (포인터(Pointer) 오버헤드) |
| **크기 조정** | 동적 배열(Dynamic Array)이면 O(1) 분할 상환 (Amortized) | 자연스럽게 유연 |

### 3.1.2 스택 — LIFO

**LIFO(Last In, First Out)** 는 마지막에 들어온 원소가 가장 먼저 나오는 자료구조이다.

```
┌─────┐
│  C  │  ← top (push / pop)
├─────┤
│  B  │
├─────┤
│  A  │  ← bottom
└─────┘
```

**연산** — `push`, `pop`, `peek`, `isEmpty` 는 모두 $O(1)$ 이다.

**실무 활용** — 함수 호출 스택(call stack), 괄호 매칭(`({[]})`), 편집기의 Undo(Ctrl+Z), DFS 구현, 수식 평가(중위 → 후위 변환) 등에 쓰인다.

### 3.1.3 큐 — FIFO

**FIFO(First In, First Out)** 는 먼저 들어온 원소가 가장 먼저 나오는 자료구조이다.

```
dequeue ←── [ A | B | C | D ] ←── enqueue
            front        rear
```

**실무 활용** — 너비 우선 탐색(BFS), 프린터 큐, CPU 스케줄링, 메시지 큐(Kafka, RabbitMQ) 등에 쓰인다.

### 3.1.4 힙 — 우선순위 큐

힙은 **힙 속성(Heap Property)** 을 만족하는 **완전 이진 트리(Complete Binary Tree)** 이다.

- **최대 힙(Max-Heap)** 에서는 부모 키가 자식 키보다 크거나 같으며, 루트에 최댓값이 있다.
- **최소 힙(Min-Heap)** 에서는 부모 키가 자식 키보다 작거나 같으며, 루트에 최솟값이 있다.

**배열 표현(1-index)**:

- 노드 $i$ 의 왼쪽 자식은 $A[2i]$ 이다.
- 노드 $i$ 의 오른쪽 자식은 $A[2i+1]$ 이다.
- 노드 $i$ 의 부모는 $A[\lfloor i/2 \rfloor]$ 이다.

**연산 복잡도**:

- `insert(x)` 와 `extractMin/Max` 는 각각 $O(\log n)$ 이다.
- `getMin/Max` 는 $O(1)$ 이다.
- `buildHeap` 은 $O(n)$ 이다. 직관적으로는 $O(n \log n)$ 일 것 같지만, 리프 노드에는 heapify 가 필요 없다는 점을 반영한 타이트 분석을 하면 $O(n)$ 으로 줄어든다.

**실무 활용** — 우선순위 큐, 힙 정렬, 다익스트라 알고리즘, 허프만 코딩 등의 기반이 된다.

## 3.2 기본 정렬 — $\Theta(n^2)$

이 절의 세 가지 정렬은 모두 점화식 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ 를 따른다. 매 단계에서 문제 크기가 1 씩만 줄어들기 때문이다.

### 3.2.1 선택 정렬

매 라운드마다 남은 부분에서 최댓값(또는 최솟값)을 찾아 올바른 위치로 보내는 정렬이다.

```
selectionSort(A[], n):
    for last = n downto 2:
        k = A[1..last] 에서 최댓값 인덱스
        swap A[k] ⇔ A[last]
```

**특징** — 최선·평균·최악이 모두 $\Theta(n^2)$ 로 입력 순서와 무관하며, 교환 횟수는 최대 $n-1$ 회이다. 불안정(Unstable) 정렬이며 추가 메모리는 $O(1)$ 만 쓴다(제자리 정렬).

### 3.2.2 버블 정렬

인접한 두 원소를 비교해 순서가 잘못되어 있으면 교환하는 정렬이다. 큰 원소가 거품처럼 끝으로 밀려 올라간다.

```
bubbleSort(A[], n):
    for last = n downto 2:
        for i = 1 to last-1:
            if A[i] > A[i+1]:
                swap A[i] ⇔ A[i+1]
```

**최적화** — `swapped` 플래그를 두어 한 패스 동안 교환이 한 번도 없으면 이미 정렬이 끝난 것이므로 조기 종료한다. 이 덕분에 최선의 경우 $\Theta(n)$ 을 달성한다.

**특징** — 최악과 평균은 $\Theta(n^2)$, 최적화된 최선은 $\Theta(n)$ 이다. `A[i] > A[i+1]` 일 때만 교환하므로 안정(Stable) 정렬이며, 제자리 정렬이다.

### 3.2.3 삽입 정렬

카드 게임에서 손에 든 카드를 정렬하는 방식과 같이, 이미 정렬된 부분에 새 원소를 올바른 위치에 삽입하는 정렬이다.

```
insertionSort(A[], n):
    for i = 2 to n:
        key = A[i]
        j = i - 1
        while j ≥ 1 and A[j] > key:
            A[j+1] = A[j]        # 오른쪽으로 밀기
            j = j - 1
        A[j+1] = key             # 올바른 위치에 삽입
```

**예시** — `[29, 10, 14, 37, 13]` 에 대해 다음과 같이 진행된다.

1. `10` 을 삽입해 `[10, 29 | 14, 37, 13]` 이 된다.
2. `14` 를 삽입해 `[10, 14, 29 | 37, 13]` 이 된다.
3. `37` 을 삽입해 `[10, 14, 29, 37 | 13]` 이 된다.
4. `13` 을 삽입해 `[10, 13, 14, 29, 37]` 이 된다.

**복잡도**:

| 경우 | 비교 횟수 | 결과 |
|---|---|---|
| 최악 (역순 정렬된 입력) | $n(n-1)/2$ | $\Theta(n^2)$ |
| 평균 | $n(n-1)/4$ | $\Theta(n^2)$ |
| 최선 (이미 정렬된 입력) | $n-1$ | $\Theta(n)$ |

**특징** — 안정 정렬이자 제자리 정렬이다. 거의 정렬된 소규모 데이터에서 실측 속도가 매우 뛰어나 Timsort 같은 실무 정렬의 내부 기저로 쓰인다.

## 3.3 고급 정렬 — $\Theta(n\log n)$

세 알고리즘 모두 점화식 $T(n) = 2T(n/2) + \Theta(n) = \Theta(n\log n)$ 을 따르며, 문제를 절반씩 분할하는 것이 공통 구조이다.

### 3.3.1 병합 정렬

**분할 정복 전략** — 배열을 절반으로 나눠 각각을 재귀적으로 정렬한 뒤, 두 정렬된 부분 배열을 합병하여 완성한다.

```
mergeSort(A, p, r):
    if p < r:
        q = ⌊(p+r)/2⌋
        mergeSort(A, p, q)
        mergeSort(A, q+1, r)
        merge(A, p, q, r)
```

**합병(Merge) 절차** — 두 정렬된 부분 배열의 선두를 비교하여 작은 쪽을 결과 배열에 복사하고 해당 포인터를 전진시키는 과정이며, 전체 비용은 $O(n)$ 이다.

**예시** — 배열 `[31, 3, 65, 73, 8, 11, 20, 29, 48, 15]` 에 대해 다음과 같이 진행된다.

```
분할:
[31, 3, 65, 73, 8]    [11, 20, 29, 48, 15]
   ↓ 재귀               ↓ 재귀
[3, 8, 31, 65, 73]    [11, 15, 20, 29, 48]
        ↓ 합병
[3, 8, 11, 15, 20, 29, 31, 48, 65, 73]
```

**특징** — 최악·평균·최선 모두 $\Theta(n\log n)$ 으로 성능이 보장되고, 안정 정렬이다. 합병을 위한 $O(n)$ 보조 배열이 필요하며, 배열에 순차 접근하므로 캐시 효율이 좋고 디스크 기반 외부 정렬에도 잘 맞는다.

### 3.3.2 퀵 정렬

퀵 정렬은 분할 정복의 뒤집힌 형태로, 분할(파티션)이 어려운 대신 합병 단계는 자명하다.

```
quickSort(A, p, r):
    if p < r:
        q = partition(A, p, r)
        quickSort(A, p, q-1)
        quickSort(A, q+1, r)

partition(A, p, r):         # Lomuto 분할
    pivot = A[r]
    i = p - 1
    for j = p to r-1:
        if A[j] ≤ pivot:
            i = i + 1
            swap A[i] ⇔ A[j]
    swap A[i+1] ⇔ A[r]
    return i + 1
```

**분할 불변식** — 파티션 루프 중 다음 세 조건이 유지된다.

- $A[p...i]$ 의 모든 원소는 피벗 이하이다.
- $A[i+1...j-1]$ 의 모든 원소는 피벗보다 크다.
- $A[j...r-1]$ 은 아직 검사되지 않은 영역이다.

**복잡도**:

| 경우 | 분할 형태 | 복잡도 |
|---|---|---|
| 최악 | 0 : n-1 (정렬된 입력에 나쁜 피벗) | $\Theta(n^2)$ |
| 평균 | 랜덤 피벗 | $\Theta(n\log n)$ |
| 최선 | 완벽한 중앙 분할 | $\Theta(n\log n)$ |

**왜 1:9 불균형도 $\Theta(n\log n)$ 인가** — 재귀 깊이가 $\log_{10/9} n = O(\log n)$ 으로 로그의 밑만 달라질 뿐이고, 각 레벨의 작업량은 $\Theta(n)$ 이므로 총 비용은 $\Theta(n\log n)$ 이 된다.

**최악 회피 방법**:

- **랜덤 피벗** — 임의 인덱스를 선택해 마지막과 스왑한 뒤 파티션한다.
- **Median-of-3** — 첫, 중간, 마지막 원소의 중앙값을 피벗으로 쓴다.
- **Introsort** — 재귀 깊이가 일정 수준을 넘으면 힙 정렬로 전환한다(C++ STL 의 `std::sort` 가 사용한다).

**특징** — 불안정 정렬이지만 제자리 정렬이며, 재귀 스택이 $O(\log n)$ 정도 필요하다. 배열에 순차 접근하므로 캐시 친화적이어서 실전에서 가장 빠른 범용 정렬로 꼽힌다.

### 3.3.3 힙 정렬

`buildHeap` 으로 배열 전체를 최대 힙으로 만든 뒤, 루트(최댓값)를 끝으로 옮기는 과정을 반복하는 정렬이다.

```
heapSort(A, n):
    buildHeap(A, n)              # O(n)
    for i = n downto 2:
        swap A[1] ⇔ A[i]
        heapify(A, 1, i-1)       # O(log i)
```

**설명** — 먼저 전체 배열을 최대 힙으로 만든다. 그런 다음 루트(최댓값)를 배열의 끝 위치로 스왑해 정렬 확정 영역에 편입시키고, 크기가 하나 줄어든 힙( `1..i-1` )에 대해 `heapify` 로 힙 속성을 복원한다. 이 과정을 반복하면 배열 끝부터 앞쪽으로 큰 값이 차례대로 놓이며 오름차순 배열이 완성된다. 추가 메모리 없이 제자리에서 동작한다.

**buildHeap 이 $O(n)$ 인 이유** — 리프 노드에 해당하는 $n/2$ 개는 heapify 가 필요 없고, 높이 $h$ 인 내부 노드는 $\approx n/2^{h+1}$ 개이며 각 heapify 는 $O(h)$ 이다. 이를 모두 합하면 $\sum_{h=1}^{\log n} \frac{n}{2^{h+1}} \cdot h = O(n)$ 이 되어 직관보다 작다.

**특징** — 최악에도 $\Theta(n\log n)$ 을 보장하며 $O(1)$ 공간만 쓰는 제자리 정렬이다. 다만 불안정 정렬이고, 트리 구조를 따라 이동하기 때문에 캐시 효율이 나쁘다.

### 3.3.4 세 정렬 비교표

| 특성 | 병합 정렬 | 퀵 정렬(Quick Sort) | 힙 정렬 |
|---|---|---|---|
| 최악 | $\Theta(n\log n)$ | $\Theta(n^2)$ | $\Theta(n\log n)$ |

| | | | |
|---|---|---|---|
| 평균 | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ |
| 공간 | $O(n)$ | $O(\log n)$ | $O(1)$ |
| 안정 | **예** | 아니오 | 아니오 |
| 캐시 | 우수 | **매우 우수** | 불량 |
| 실전 속도 | 빠름 | **가장 빠름** | 보통 |

**선택 가이드:**

- 안정성이 필수이면 병합 정렬을 쓴다.
- 메모리 제약이 있고 최악 성능 보장이 필요하면 힙 정렬을 쓴다.
- 일반적으로 가장 빠른 정렬이 필요하면 랜덤 피벗을 사용한 퀵 정렬을 쓴다.

## 3.4 비교 기반 정렬의 하한

**정리** — 임의의 비교 기반 정렬은 최악의 경우 $\Omega(n\log n)$ 번의 비교가 필요하다.

**증명 스케치 (결정 트리 모델):**

- $n!$ 개의 가능한 순열에 대응하는 $n!$ 개의 리프가 있어야 한다.
- 높이가 $h$ 인 이진 결정 트리의 리프 수는 최대 $2^h$ 이다.
- 따라서 $h \geq \log_2(n!) = \Theta(n\log n)$ (Stirling 근사에 의해) 이 된다.

비교만을 사용하는 정렬은 이 하한을 절대 깰 수 없다. 계수 정렬이나 기수 정렬은 비교 대신 값의 범위나 자릿수 같은 추가 정보를 활용하므로 이 하한을 우회할 수 있다.

## 3.5 선형 시간 정렬 — 비교 기반 하한 우회

### 3.5.1 계수 정렬

값이 $[0, k]$ 범위에 있을 때 $\Theta(n + k)$ 시간에 정렬할 수 있다.

```
countingSort(A, n, k):
    C[0..k] = 0
    for j = 1 to n: C[A[j]] += 1      # 발생 횟수
    for i = 1 to k: C[i] += C[i-1]    # 누적합
    for j = n downto 1:               # 역순 순회 (안정성)
        B[C[A[j]]] = A[j]
        C[A[j]] -= 1
```

**설명** — (1) 각 값 $v$ 가 몇 번 등장하는지 센 뒤, (2) 누적합으로 변환하면 `C[v]` 가 "**값 $v$ 이하 원소의 총 개수**" = "출력 배열에서 값 $v$ 의 마지막 위치" 가 된다. (3) 입력을 **뒤에서 앞** 으로 순회하며 각 원소를 `B[C[A[j]]]` 에 배치하고 `C` 를 하나씩 감소. 비교 연산 없이 정수 값의 범위 정보만으로 정렬이 완성되어 $\Theta(n+k)$.

**역순 순회가 안정성을 보장하는 이유** — 같은 값을 가진 여러 원소 중 뒤에 있던 것이 결과 배열의 뒤쪽에 배치되어 입력 순서가 그대로 보존된다. 앞에서부터 순회하면 순서가 역전되어 불안정 정렬이 된다.

### 3.5.2 기수 정렬

자릿수별로 안정 정렬(보통 계수 정렬)을 반복하는 방식이며, 반드시 **최하위(LSD)부터 최상위(MSD)** 순서로 진행한다.

```
radixSort(A, d):
    for digit = 1 to d:
        A = stable_sort_by_digit(A, digit)
```

**설명** — $d$ 자릿수 정수 키를 **최하위 자릿수부터 최상위 자릿수** 로 정렬. 각 자리별 정렬은 **안정 정렬**(보통 계수 정렬)을 사용. 핵심 통찰 — 이전 자리까지 정렬된 상태가 안정성 덕분에 유지되므로, 상위 자리 정렬 후에도 하위 자리는 이미 정렬되어 있다. 결과적으로 모든 자리가 정렬된 최종 배열이 만들어진다.

LSD 순서로 진행해야 이전 자리의 정렬 순서가 안정 정렬 덕분에 유지된다.

**복잡도** — $\Theta(d(n+k))$ 이며, $d$ 와 $k$ 가 상수이면 $\Theta(n)$ 이 된다.

---

## 3장 정리

3주차의 큰 그림은 다음과 같다. 자료구조(배열·연결 리스트·스택·큐·힙)는 알고리즘이 사용하는 도구이다. 정렬은 $\Theta(n^2)$ 의 기본 정렬, $\Theta(n\log n)$ 의 고급 정렬, $\Theta(n)$ 의 선형 시간 정렬로 나뉜다. 비교 기반 하한 $\Omega(n\log n)$ 은 결정 트리 모델로 증명되며, 계수 정렬과 기수 정렬은 비교를 우회해 이 하한을 깬다. 각 정렬의 최악·평균·최선, 안정성, 제자리 여부가 시험에 자주 출제된다.

---

# 4장. 분할 정복

## 4.1 분할 정복 패러다임

**세 단계:**

1. **분할(Divide)** — 문제를 더 작은 부분 문제들로 나눈다.
2. **정복(Conquer)** — 각 부분 문제를 재귀적으로 해결한다.
3. **결합(Combine)** — 부분 해들을 합쳐 원래 문제의 해를 만든다.

**필수 조건** — 부분 문제가 서로 독립적이어야 한다. 부분 문제가 겹친다면 분할 정복 대신 DP 를 써야 한다.

**전형적 점화식** — $T(n) = aT(n/b) + f(n)$ 형태이며 마스터 정리로 푼다.

## 4.2 대표 알고리즘 카탈로그

| 알고리즘 | 분할 | 정복 | 결합 | 점화식 | 시간 |
|---|---|---|---|---|---|
| **병합 정렬** | 반 쪼갬 | 각 반 정렬 | 합병 | $2T(n/2) + \Theta(n)$ | $\Theta(n\log n)$ |
| **이진 탐색** | 중간 비교 | **한쪽만** 재귀 | 자명 | $T(n/2) + O(1)$ | $\Theta(\log n)$ |
| **퀵 정렬** (평균) | 파티션 | 양쪽 재귀 | 자명 | $2T(n/2) + \Theta(n)$ | $\Theta(n\log n)$ |
| **퀵 정렬** (최악) | 불균형 | — | — | $T(n-1) + \Theta(n)$ | $\Theta(n^2)$ |
| **랜덤 선택** | 파티션 | **한쪽만** | 자명 | $T(3n/4) + \Theta(n)$ | 평균 $\Theta(n)$ |
| **중앙값의 중앙값** | 5 개씩 그룹 | 양쪽 | 파티션 | $T(n/5) + T(7n/10) + \Theta(n)$ | $\Theta(n)$ 최악 |
| **최근접 쌍 (Closest Pair)** | x 로 분할 | 각 반 | 스트립 | $2T(n/2) + O(n\log n)$ | $O(n\log^2 n)$ |

## 4.3 선택 문제

**문제 정의** — 정렬되지 않은 배열에서 $i$ 번째로 작은 원소를 찾는다. 전체 배열을 정렬($O(n \log n)$) 하지 않고도 가능한지가 관건이다.

### 4.3.1 랜덤 선택 — 기대 $\Theta(n)$

```
select(A, p, r, i):
    if p == r: return A[p]
    q = partition(A, p, r)           # 퀵 정렬의 파티션
    k = q - p + 1
    if i == k: return A[q]
    elif i < k: return select(A, p, q-1, i)
    else:       return select(A, q+1, r, i-k)
```

**설명** — 퀵 정렬의 `partition` 을 재사용해 피벗을 최종 위치에 놓고 그 순위 $k$ (A[p..r] 내에서 피벗이 몇 번째 작은 원소인가)를 구함. $i = k$ 면 피벗이 정답. $i < k$ 면 원하는 원소가 **왼쪽 부분** 에 있으므로 왼쪽만 재귀. $i > k$ 면 오른쪽만 재귀하되, 이미 $k$ 개(왼쪽+피벗)를 건너뛰었으므로 찾을 순위는 $i - k$ 로 보정. 양쪽을 모두 재귀하지 않아 평균 선형 시간.

퀵 정렬과 달리 양쪽이 아니라 한쪽만 재귀 호출하므로 평균적으로 훨씬 빠르다.

**분석** — 평균적으로 문제 크기가 $3n / 4$ 이하로 감소하므로 다음이 성립한다.

$$T(n) \leq T(3n / 4) + \Theta(n)$$

기하급수로 전개하면

$$T(n) \leq cn + c \cdot \frac{3n}{4} + c \cdot \left(\frac{3}{4}\right)^2 n + \cdots = 4cn = \Theta(n)$$

이 된다.

## 4.3.2 중앙값의 중앙값 — 최악 $\Theta(n)$

**아이디어** — 항상 균형 잡힌 피벗을 보장하는 방식이다.

1. 원소를 5개씩 그룹으로 나누어 $\lceil n / 5 \rceil$ 개의 그룹을 만든다.
2. 각 그룹의 중앙값을 구한다.
3. 그 중앙값들의 중앙값 $M$ 을 재귀적으로 찾는다.
4. $M$ 을 피벗으로 삼아 파티션한다.
5. 원하는 쪽에 대해 재귀 호출한다.

**보장** — 최소 $3n / 10$ 개 원소가 $M$ 이하이고 최소 $3n / 10$ 개가 $M$ 이상이므로, 재귀 크기는 최대 $7n / 10$ 이 된다.

**점화식** — $T(n) = T(n / 5) + T(7n / 10) + \Theta(n)$ 이며, $1 / 5 + 7 / 10 = 9 / 10 < 1$ 이므로 기하급수 합이 수렴하여 $\Theta(n)$ 이 된다.

**왜 5개씩인가** — 3개로 묶으면 $1/3 + 2/3 = 1$ 이 되어 수렴하지 않는다. 5가 수렴을 보장하는 최소 크기이다.

## 4.4 최근접 점 쌍

**문제 정의** — 평면의 $n$ 개 점 중 거리가 최소인 두 점을 찾는다.

**무차별 대입** — 모든 쌍을 확인하면 $O(n^2)$ 이다. 더 나은 방법이 있는가?

**분할 정복 전략**:

1. 점들을 x 좌표 기준으로 정렬한다(전처리 $O(n \log n)$).
2. 중앙 x 를 기준으로 좌·우로 분할한다.
3. 각 절반을 재귀 호출하여 $d_L, d_R$ 을 구한다.
4. $d = \min(d_L, d_R)$ 로 설정한다.
5. 분할선에서 $d$ 이내의 점들로 스트립을 구성하고, 두 반에 걸친 쌍이 $d$ 보다 가까운지 검사한다.

**스트립의 기하학적 성질** — $d \times 2d$ 크기의 직사각형 안에서 서로 $d$ 이상 떨어진 점은 최대 8개이다. 따라서 스트립을 y 좌표로 정렬한 뒤, 각 점은 최대 7개의 y 방향 이웃만 검사하면 충분하므로 스트립 검사 비용은 $O(n)$ 이다.

**복잡도** — 점화식 $T(n) = 2T(n/2) + O(n\log n)$ 에서 $O(n\log^2 n)$ 이 되며, 스트립 정렬을 보존하는 최적화를 쓰면 $O(n\log n)$ 까지 줄일 수 있다.

## 4.5 분할 정복이 실패하는 경우

순진한 재귀로 피보나치를 구현하면 다음과 같다.

```
def fib(n):
    if n <= 2: return 1
    return fib(n-1) + fib(n-2)
```

점화식은 $T(n) = T(n-1) + T(n-2) + O(1)$ 이며, 해는 $O(\phi^n) \approx O(1.618^n)$ 으로 지수 시간이다.

**원인** — 같은 부분 문제가 여러 번 반복 계산되는 중복 부분 문제(Overlapping Subproblems)에 있다. 예를 들어 $F(3)$ 이 트리 곳곳에서 반복된다.

**해결** — 이런 문제는 DP(6주차)로 해결한다.

**분할 정복과 DP 의 구분**:

- 부분 문제가 서로 **독립적** 이면 분할 정복을 쓴다.

- 부분 문제가 **겹친다면** DP 를 써야 한다.

---

## 4장 정리

4주차의 큰 그림은 다음과 같다. 분할 정복은 독립적인 부분 문제들을 재귀적으로 풀어 결합하는 패러다임이다. 대표 알고리즘으로는 병합 정렬($\Theta(n \log n)$), 이진 탐색($\Theta(\log n)$), 퀵 정렬(평균 $\Theta(n \log n)$), 랜덤 선택($\Theta(n)$), 중앙값의 중앙값($\Theta(n)$ 최악), 최근접 쌍($O(n \log^2 n)$) 이 있다. 부분 문제가 겹치는 피보나치 같은 문제는 DP 로 해결한다.

---

# 5장. 그리디 알고리즘

## 5.1 그리디 패러다임

그리디 알고리즘은 매 단계에서 지금 당장 가장 좋아 보이는 선택(지역 최적)을 한다. 한 번 내린 선택을 되돌리지 않으며 백트래킹도 하지 않는다.

```
Greedy(C):
    S = {}
    while C 가 비지 않고 S 가 미완성:
        x = C 에서 가장 좋아 보이는 원소
        C = C − {x}
        if S ∪ {x} 가 실행 가능:
            S = S ∪ {x}
    return S
```

**설명** — 일반적인 그리디 프레임워크. 후보 집합 $C$ 에서 **어떤 기준(예: 가치/무게, 종료 시간, 간선 가중치)에 따라 가장 좋아 보이는** 원소 $x$ 를 하나씩 꺼내 해 집합 $S$ 에 추가. **이미 내린 선택을 되돌리지 않는다(백트래킹 없음)**. 실행 가능성 검사로 제약(예: 사이클 생성, 용량 초과)을 회피. 이 단순한 루프가 최적해를 보장하려면 문제가 **그리디 선택 속성** 과 **최적 부분 구조** 를 만족해야 함.

## 5.2 정확성을 위한 두 필수 조건

그리디 알고리즘이 최적해를 보장하려면 다음 두 조건이 모두 성립해야 한다.

1. **그리디 선택 속성(Greedy-Choice Property)** — 어떤 최적해는 반드시 그리디 선택을 포함한다. 이를 증명하는 표준 기법은 교환 논증(Exchange Argument)이다. 최적해 안의 비-그리디 선택을 그리디 선택으로 교환해도 최적성이 유지됨을 보이면 된다.

2. **최적 부분 구조(Optimal Substructure)** — 그리디 선택을 하고 남은 부분 문제의 최적해가 전체 최적해의 일부가 된다.

**실전 원칙** — 먼저 그리디를 시도하고, 반례가 나오면 DP 로 전환한다.

## 5.3 그리디 실패 사례

| 문제 | 그리디 선택 | 결과 | 최적해 |
|---|---|---|---|
| 동전 {1, 3, 4}, 금액 6 | 4 + 1 + 1 = 3개 | 실패 | 3 + 3 = 2개 |
| 동전 {16, 10, 5, 1}, 금액 20 | 16 + 1 + 1 + 1 + 1 = 5개 | 실패 | 10 + 10 = 2개 |
| 0/1 배낭 (가치/무게 정렬) | 비율 큰 것부터 선택 | 용량 낭비 | DP 필요 |
| 이진 트리 최대 경로 합 | 큰 자식으로 이동 | 지역 최적만 달성 | 전역 최적을 놓친다 |

한국 원화(500, 100, 50, 10) 같은 표준 화폐는 각 단위가 다음 작은 단위의 배수이므로 그리디가 최적이지만, 비표준 단위 집합에서는 반드시 반례를 의심해야 한다.

## 5.4 그리디 알고리즘 카탈로그

### 5.4.1 동전 거스름돈 (표준 화폐)

큰 화폐부터 선택하는 단순한 규칙이며, 표준 화폐에서는 $O(k)$ 시간에 최적해를 얻는다.

### 5.4.2 분할 가능 배낭

물건을 쪼갤 수 있는 배낭 문제는 그리디로 최적해를 얻을 수 있다.

**전략** — 단위 무게당 가치($v_i / w_i$)를 기준으로 내림차순 정렬한 뒤 순서대로 배낭에 담는다.

```
FractionalKnapsack(items, C):
    각 물건의 v/w 계산, 내림차순 정렬
    남은 용량 w = C
    for 각 물건 x (정렬 순):
        if w ≥ x.weight: 전부 가져감
        else: 남은 용량만큼 분수 가져감
```

**예시** — 용량이 40이고 물건이 백금(10kg, $60), 금(15kg, $75), 은(25kg, $10), 주석(20kg, $2)일 때, 비율은 6, 5, 0.4, 0.1 이다.

| 순서 | 물건 | 처리 | 누적 가치 |
|---|---|---|---|
| 1 | 백금(10kg) | 전부 담는다 | $60 |
| 2 | 금(15kg) | 전부 담는다 | $135 |
| 3 | 은(25kg) | 남은 용량만큼 0.6 만 담는다 | $141 |

**시간 복잡도** — 정렬 단계가 지배하여 $O(n\log n)$ 이다.

0/1 배낭처럼 물건을 쪼갤 수 없는 경우에는 그리디가 실패하며 DP 를 써야 한다.

### 5.4.3 활동 선택

**문제 정의** — 한 기계 위에서 서로 겹치지 않는 활동의 최대 개수를 선택한다.

**그리디 전략** — 가장 빠른 종료 시간(Earliest Finish Time, EFT)을 가진 활동부터 선택한다.

```
ActivitySelection(jobs):
    종료 시간 오름차순 정렬
    selected = {첫 활동}
    last_finish = 첫 활동 종료
    for 나머지 j:
        if j.start ≥ last_finish:
            selected += {j}
            last_finish = j.finish
    return selected
```

**설명** — 종료 시간 기준으로 정렬 후 순회하며, 현재까지 선택된 마지막 활동의 종료 시간 (`last_finish`)보다 **시작 시간이 늦거나 같은** 활동만 선택. 선택할 때마다 `last_finish` 를 갱신. 정렬 $O(n\log n)$ + 선형 스캔 $O(n)$ = $O(n\log n)$. 전체 활동을 한 번씩 확인하므로 간결하다.

**왜 EFT 인가** — 가장 빨리 끝나는 활동을 먼저 선택해야 이후 활동들을 위한 시간이 최대로 남기 때문이다.

**교환 논증 증명** — 최적해 $O$ 와 그리디 해 $G$ 를 비교해 보자. $G$ 의 첫 활동 $g_1$ 은 모든 활동 중 가장 일찍 끝나므로 $\text{finish}(g_1) \leq \text{finish}(o_1)$ 이다. 귀납적으로 $G$ 가 $O$ 만큼 또는 그 이상의 활동을 선택함을 보일 수 있다.

### 5.4.4 작업 스케줄링 (활동 선택과 혼동 주의)

**문제 정의** — 모든 작업을 수행하되 사용되는 기계의 수를 최소화한다.

**그리디 전략** — 가장 빠른 시작 시간(Earliest Start Time, EST)을 가진 작업부터 배정한다.

**활동 선택과의 구분**:

| 구분 | 작업 스케줄링 | 활동 선택 |
|---|---|---|
| 목표 | 기계 수 최소화 | 활동 수 최대화 |
| 그리디 기준 | EST (시작 시간) | EFT (종료 시간) |

### 5.4.5 허프만 코딩 — 최적 접두 코드

**문제 정의** — 문자 빈도를 바탕으로 가변 길이 이진 코드를 할당해 파일을 압축한다.

**접두사 성질(Prefix Property)** — 어떤 코드도 다른 코드의 접두사가 아닌 성질이다. 이 성질 덕분에 구분자 없이도 디코딩이 가능하다.

**알고리즘 개요** — 빈도 기반 최소 힙에서 가장 빈도가 낮은 두 노드를 반복해서 합친다.

```
Huffman(C):
    Q = 빈도 기반 최소 힙 (리프 = 각 문자)
    while |Q| ≥ 2:
        A = extract-min(Q)
        B = extract-min(Q)
        N = new node with freq = A.freq + B.freq
        N.left = A, N.right = B
        insert(Q, N)
    return Q의 루트
```

**설명** — 각 문자를 리프 노드로 하는 최소 힙에서, **가장 빈도가 낮은 두 노드** 를 꺼내 합쳐진 빈도를 가진 새 내부 노드로 만들어 다시 힙에 넣는다. 합쳐지는 노드는 트리에서 깊게 배치되어 긴 코드를, 나머지는 짧은 코드를 갖는다. $n-1$ 번 반복하면 단일 루트의 이진 트리가 완성. 루트→리프 경로의 0/1 나열이 각 문자의 **접두 코드**. 복잡도는 힙 연산 $O(\log n)$ 을 $n$ 번 수행해 $O(n\log n)$.

**예시** — 문자 빈도가 A:450, T:90, G:120, C:270 일 때 트리는 다음과 같이 구성된다.

1. 가장 빈도가 낮은 T(90) 과 G(120) 을 합쳐 빈도 210 의 새 노드를 만든다.
2. 다음으로 낮은 210 과 C(270) 을 합쳐 빈도 480 의 노드를 만든다.
3. 마지막으로 A(450) 과 480 을 합쳐 빈도 930 의 루트가 완성된다.

**할당된 코드**:

- A 는 `0` (1비트)
- C 는 `10` (2비트)
- T 는 `110` (3비트)
- G 는 `111` (3비트)

**비트 수 계산** — $450 \cdot 1 + 270 \cdot 2 + 90 \cdot 3 + 120 \cdot 3 = 1{,}620$ 비트이다. 8비트 ASCII 로 저장했다면 $930 \cdot 8 = 7{,}440$ 비트가 필요했으므로 압축률은 약 21.8% 이다.

**복잡도** — 힙에서 $n-1$ 번 extract 와 insert 를 수행하므로 $O(n\log n)$ 이다.

**최적성 증명** — 교환 논증으로 증명된다. 핵심은 최적 트리에서도 최저 빈도 두 문자가 반드시 같은 부모를 공유하는 형제 리프여야 한다는 점이다.

## 5.5 최소 신장 트리 (MST)

**문제 정의** — 연결된 가중 그래프에서 모든 정점을 연결하는 가중치 합 최소의 트리($n-1$ 개의 간선)를 구한다.

### 5.5.1 크루스칼 — 간선 중심

```
KruskalMST(G):
    sort 간선 by 가중치 오름차순
    T = {}
    Union-Find 초기화
    for 각 간선 (u, v) (오름차순):
        if find(u) ≠ find(v):           # 사이클 없음
            T += {(u, v)}
            union(u, v)
    return T
```

**설명** — 간선 중심 접근. 모든 간선을 가중치 오름차순으로 정렬한 뒤, 순서대로 보며 **사이클을 만들지 않으면 트리에 편입**. 사이클 검사는 유니온-파인드의 `find` 로 두 끝점이 이미 같은 연결 성분에 속하는지를 확인. 포함하면 `union` 으로 두 성분을 합친다. $n-1$ 개 간선을 선택하면 MST 완성. **작은 숲들이 점차 병합** 되어 하나의 트리가 되는 과정.

**자료구조** — 유니온-파인드(Union-Find)를 사용하며, 사이클 감지를 거의 상수 시간인 $O(\alpha(n))$ 에 처리한다.

**복잡도** — 정렬이 지배하여 $O(m\log m)$ 이다.

**적합한 상황** — 간선 수가 적은 희소 그래프(Sparse Graph)에 유리하다.

### 5.5.2 프림 — 정점 중심

```
PrimMST(G, s):
    D[v] = ∞ for all v; D[s] = 0
    T = {s}
    while |T| < n:
        u = T 밖에서 D[u] 최소 정점
        T += {u}
        for u 의 이웃 v (v ∉ T):
            if weight(u,v) < D[v]:
                D[v] = weight(u,v)
                parent[v] = u
    return MST 간선들
```

**설명** — 정점 중심 접근. 시작 정점 $s$ 에서 출발해 **하나의 트리를 키워가는** 방식. 각 정점 $v$ 에 대해 $D[v]$ = "현재 트리에서 $v$ 까지의 최소 연결 간선 가중치" 를 유지. 매 반복에서 트리에 아직 속하지 않은 정점 중 $D$ 가 최소인 $u$ 를 트리에 편입하고, $u$ 의 이웃들에 대해 $D$ 값을 더 작게 갱신. 모든 정점이 트리에 포함될 때까지 반복.

**자료구조** — 우선순위 큐를 사용한다.

**복잡도** — 배열 기반 구현은 $O(n^2)$, 이진 힙 기반은 $O(m\log n)$ 이다.

**적합한 상황** — 간선이 많은 밀집 그래프(Dense Graph)에 유리하다.

### 5.5.3 크루스칼과 프림의 비교

| 구분 | 크루스칼 | 프림 |
|---|---|---|
| 접근 | 간선 중심 | 정점 중심 |
| 자료구조 | 유니온-파인드 | 우선순위 큐 |
| 복잡도 | $O(m\log m)$ | $O(n^2)$ 또는 $O(m\log n)$ |
| 적합 그래프 | 희소 | 밀집 |
| 구축 과정 | 여러 숲이 병합되어 트리가 된다 | 한 트리가 점차 성장한다 |

## 5.6 다익스트라 — 최단 경로

**문제 정의** — 출발 정점 $s$ 에서 다른 모든 정점까지의 최단 경로를 구한다. 단, 간선 가중치는 음이 아니어야 한다.

**알고리즘**:

```
Dijkstra(G, s):
    D[v] = ∞; D[s] = 0; S = {}
    while S ≠ V:
        u = S 밖에서 D[u] 최소 정점
        S += {u}
        for u 의 이웃 v (v ∉ S):        # 간선 완화 (Relax)
            if D[u] + w(u,v) < D[v]:
                D[v] = D[u] + w(u,v)
                parent[v] = u
```

**설명** — $s$ 로부터의 최단 거리 $D[\cdot]$ 를 유지하며, 매 단계 **아직 확정되지 않은 정점 중 $D$ 가 최소인 $u$ 를 확정**(집합 $S$ 에 편입). 그 후 $u$ 의 이웃 $v$ 에 대해 "$u$ 를 경유하는 경로가 현재 $D[v]$ 보다 짧으면 갱신" 하는 **간선 완화(Relaxation)** 를 수행. 음이 아닌 가중치 가정 덕분에 $S$ 에 편입된 정점의 $D$ 값은 이후 바뀌지 않는다는 그리디 불변식이 성립.

**그리디 불변식** — $u \in S$ 이면 $D[u]$ 는 $s$ 에서 $u$ 까지의 실제 최단 거리이다.

**음수 간선에서 실패하는 이유** — 다익스트라는 일단 확정된 $D[u]$ 가 이후에 바뀌지 않는다고 가정한다. 하지만 음수 간선이 있으면 나중에 더 짧은 경로가 발견될 수 있어 이 가정이 깨진다. 이런 경우에는 벨만-포드 알고리즘($O(VE)$)을 써야 한다.

**프림 알고리즘과의 유사성**:

| 구분 | 프림 | 다익스트라 |
|---|---|---|
| 불변식 | $T$ 는 MST 에 포함된 정점 집합이다. | $S$ 는 최단 거리가 확정된 정점 집합이다. |
| $D[v]$ 의 의미 | 현재 트리까지의 최소 연결 간선 가중치이다. | 출발점 $s$ 에서의 최단 거리이다. |
| 동작 패턴 | 최솟값 추출 → 트리 확장 → 이웃 갱신 | 최솟값 추출 → 거리 확정 → 간선 완화 |

구조적으로 거의 동일하며, 차이는 "이웃 갱신" 의 의미뿐이다.

---

## 5장 정리

5주차의 큰 그림은 다음과 같다. 그리디 알고리즘은 매 단계에서 지역 최적을 선택한다. 정확성이 보장되려면 그리디 선택 속성(교환 논증으로 증명)과 최적 부분 구조가 모두 필요하다. 대표 알고리즘은 활동 선택(EFT), 분할 가능 배낭(가치/무게 비율), 허프만 코딩(최저 빈도 병합), 크루스칼과 프림의 MST, 다익스트라 최단 경로이다. 그리디가 실패하는 동전 거스름돈(임의 단위)과 0/1 배낭 문제는 DP로 풀어야 한다.

---

# 6장. 동적 계획법

## 6.1 DP 의 본질

동적 계획법(Dynamic Programming, DP)은 중복되는 부분 문제를 한 번만 풀어 저장하고 재사용함으로써 지수적 재귀를 다항식으로 축소하는 기법이다.

"프로그래밍" 이라는 용어는 컴퓨터 프로그래밍이 아니라 수학적 테이블화(tabular optimization)를 뜻하며, 1950 년대 리처드 벨만(Richard Bellman)이 도입한 용어이다.

## 6.2 필수 두 조건

1. **최적 부분 구조** — 전체 최적해가 부분 문제의 최적해로 구성된다.
2. **중복 부분 문제** — 같은 부분 문제가 재귀 구조에서 여러 번 반복 계산된다.

이 두 조건이 다른 패러다임의 조건과 어떻게 다른지 비교하면 다음과 같다.

- 분할 정복은 최적 부분 구조만 필요하고 부분 문제는 서로 독립적이어야 한다.
- 그리디는 최적 부분 구조와 그리디 선택 속성이 모두 필요하다.
- DP 는 최적 부분 구조와 중복 부분 문제가 함께 있을 때 쓴다.

## 6.3 두 가지 구현 전략

### 6.3.1 메모이제이션 — 하향식

재귀 호출에 캐시를 결합한 방식이며, 호출 구조는 자연스럽게 DAG 형태로 부분 문제를 탐색한다.

```python
def fib_memo(n, memo=None):
    if memo is None: memo = {}
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
```

**설명** — 재귀 구조는 그대로 두고 **캐시(딕셔너리 `memo`)** 를 추가. 각 `fib_memo(k)` 호출 시 먼저 캐시 조회 → 있으면 즉시 반환, 없으면 재귀로 계산 후 저장. 결과적으로 각 $k$ 값은 **정확히 한 번만** 계산되어 $O(2^n) \to O(n)$ 으로 축소.

**장점** — 실제로 필요한 부분 문제만 계산하고, 구현이 직관적이다. **단점** — 재귀 호출 스택 오버헤드가 있고 깊이 제한에 부딪힐 수 있다.

### 6.3.2 테이블화 — 상향식

반복문으로 기저 조건부터 차례대로 테이블을 채우는 방식이다.

```python
def fib_tab(n):
    if n <= 1: return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

**설명** — 재귀 없이 **0부터 $n$ 까지 반복문** 으로 $dp[i]$ 를 순서대로 채움. 각 $dp[i]$ 는 이미 계산된 $dp[i-1]$, $dp[i-2]$ 만 참조하므로 의존성 역순으로 가기만 하면 OK. 스택 오버헤드 없고 배열 순차 접근이라 캐시 효율 극대화.

**장점** — 재귀 오버헤드가 없고 배열 순차 접근이라 캐시 효율이 좋으며 실측 속도가 빠르다. **단점** — 모든 부분 문제를 계산해야 하고 계산 순서를 명시적으로 결정해야 한다.

### 6.3.3 비교

| | 메모이제이션(Memoization) | 테이블화(Tabulation) |
|---|---|---|
| 방향 | 하향식(Top-down) (큰 → 작은) | 상향식(Bottom-up) (작은 → 큰) |
| 구현 | 재귀 + 캐시 | 반복문 |
| 부분 문제 | 필요한 것만 (lazy) | 모두 계산 |
| | | |

| | | |
|---|---|---|
| 오버헤드 | 재귀 스택, 조회 | 타이트 루프 |
| 실전 | 쉬움 | 더 빠름 |

## 6.4 DP 6단계 레시피

모든 DP 문제는 다음 여섯 단계로 접근한다.

1. **부분 문제 정의** — `dp[i, j, ...]` 가 정확히 무엇을 의미하는지 정한다.
2. **점화식 작성** — 해당 부분 문제를 더 작은 부분 문제로 어떻게 표현할지 정한다.
3. **기저 조건 식별** — 재귀가 자명하게 끝나는 경계값을 정한다.
4. **계산 순서 결정** — 의존성을 만족하도록 부분 문제를 푸는 순서를 정한다.
5. **답 추출** — 테이블의 어느 셀에 최종 답이 저장되는지 확인한다.
6. **(선택) 역추적** — 값뿐만 아니라 실제 해를 복원해야 한다면 역추적 경로를 설계한다.

## 6.5 대표 DP 문제 카탈로그

### 6.5.1 피보나치 — 예열

**부분 문제** — $f[i]$ 는 $i$ 번째 피보나치 수이다. **점화식** — $f[i] = f[i-1] + f[i-2]$ 이고, 기저 조건은 $f[0] = 0$, $f[1] = 1$ 이다. **복잡도** — 시간은 $O(n)$, 공간은 $O(n)$ 이다. 두 변수만 유지하면 공간을 $O(1)$ 로 줄일 수 있다.

**실측 비교 ($n = 30$):**

| 방법 | 시간 |
|---|---|
| 순진 재귀 | 약 150 ms, $O(2^n)$ |
| 메모이제이션 | 약 0.005 ms, $O(n)$ |
| 테이블화 | 약 0.003 ms, $O(n)$ |

### 6.5.2 최장 공통 부분 수열 (LCS)

**문제 정의** — 두 문자열 $X, Y$ 의 공통 부분 수열 중 가장 긴 것을 구한다.

**부분 수열 vs 부분 문자열** — 부분 수열은 연속할 필요가 없다. 예를 들어 `"ABCBDAB"` 와 `"BDCABA"` 의 LCS 는 `"BCAB"` (길이 4) 이다.

**부분 문제** — $c[i][j]$ 는 $X[1..i]$ 와 $Y[1..j]$ 의 LCS 길이이다.

**점화식**: $$ c[i,j] = \begin{cases} 0 & i = 0 \text{ 또는 } j = 0 \ c[i-1, j-1] + 1 & X[i] = Y[j] \ \max(c[i-1, j], c[i, j-1]) & X[i] \neq Y[j] \end{cases} $$

**예시** — $X$ = "ABCBDAB", $Y$ = "BDCAB" 의 DP 테이블은 다음과 같다.

```
     ""  B  D  C  A  B
""    0  0  0  0  0  0
A     0  0  0  0  1  1
B     0  1  1  1  1  2
C     0  1  1  2  2  2
B     0  1  1  2  2  3
D     0  1  2  2  2  3
A     0  1  2  2  3  3
B     0  1  2  2  3  4
```

**표 읽는 법** — 행은 $X$ 의 접두사( `A` , `AB` , `ABC` 등), 열은 $Y$ 의 접두사( `B` , `BD` 등) 를 나타낸다. 각 셀은 해당 두 접두사의 LCS 길이이다. 0행과 0열은 빈 문자열에 해당하므로 값이 0 이다. 내부 셀은 점화식에 따라 좌측 위·왼쪽·위 셀을 참조해 채우며, 오른쪽 아래 모서리 셀이 최종 답이다.

**정답** — $c[7][5] = 4$ 이며 LCS 의 한 예는 "BCAB" 이다.

**역추적** — $c[m][n]$ 에서 시작해 문자가 일치하면 대각선으로 이동하면서 문자를 포함하고, 일치하지 않으면 값이 큰 쪽(위 또는 왼쪽)으로 이동한다.

**복잡도** — 시간과 공간 모두 $O(mn)$ 이다.

**응용** — Git diff, DNA 서열 정렬, 편집 거리의 변형 문제 등에 쓰인다.

### 6.5.3 편집 거리 (Edit Distance, Levenshtein)

**문제 정의** — 문자열 $X$ 를 $Y$ 로 변환하는 최소 연산 수를 구한다. 허용되는 연산은 삽입, 삭제, 교체이며 비용은 각각 1 이다.

**부분 문제** — $E[i][j]$ 는 $X[1..i]$ 를 $Y[1..j]$ 로 바꾸는 최소 연산 수이다.

**점화식**: $$ E[i,j] = \begin{cases} j & i = 0 \quad (\text{j 개 삽입}) \ i & j = 0 \quad (\text{i 개 삭제}) \ E[i-1, j-1] & X[i] = Y[j] \ 1 + \min(E[i-1, j], E[i, j-1], E[i-1, j-1]) & \text{else} \end{cases} $$

- $E[i-1, j] + 1$ 은 삭제 연산을 뜻한다.
- $E[i, j-1] + 1$ 은 삽입 연산을 뜻한다.
- $E[i-1, j-1] + 1$ 은 교체 연산을 뜻한다.

**예시** — "kitten" 을 "sitting" 으로 바꾸는 편집 거리는 다음 테이블로 구할 수 있다.

```
     ""  s  i  t  t  i  n  g
  ""  0  1  2  3  4  5  6  7
  k   1  1  2  3  4  5  6  7
  i   2  2  1  2  3  4  5  6
  t   3  3  2  1  2  3  4  5
  t   4  4  3  2  1  2  3  4
  e   5  5  4  3  2  2  3  4
  n   6  6  5  4  3  3  2  3
```

**표 읽는 법** — 행은 $X$(= `kitten`)의 접두사, 열은 $Y$(= `sitting`)의 접두사를 나타낸다. 각 셀은 두 접두사 사이의 편집 거리이다. 0행의 $j$ 는 빈 문자열을 $Y[1..j]$ 로 만드는 데 드는 $j$ 회의 삽입을, 0열의 $i$ 는 $X[1..i]$ 를 빈 문자열로 만드는 데 드는 $i$ 회의 삭제를 뜻한다. 내부 셀은 문자가 같으면 대각선 값을 그대로 따오고, 다르면 위·왼쪽·대각선 이웃의 최솟값에 1 을 더한다.

**정답** — $E[6][7] = 3$ 이며, 연산 순서는 k 를 s 로 교체, e 를 i 로 교체, 끝에 g 를 삽입이다.

**복잡도** — 시간과 공간 모두 $O(mn)$ 이다.

### 6.5.4 0/1 배낭

**문제 정의** — $n$ 개의 물건(각 무게 $w_i$, 가치 $v_i$)과 배낭 용량 $C$ 가 주어진다. 물건을 쪼갤 수 없다는 조건 아래 총 가치를 최대화한다.

**부분 문제** — $K[i, w]$ 는 물건 $1..i$ 까지를 고려하고 용량이 $w$ 일 때의 최대 가치이다.

**점화식**: $$ K[i, w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \ K[i-1, w] & w_i > w \ \max(K[i-1, w], K[i-1, w - w_i] + v_i) & \text{else} \end{cases} $$

**의사결정** — 각 물건에 대해 배낭에 포함할지 제외할지를 결정한다.

**예시** — 물건의 (무게, 가치)가 $(2, 12), (1, 10), (3, 20), (2, 15)$ 이고 용량 $C = 5$ 일 때 DP 테이블은 다음과 같다.

```
      0   1   2   3   4   5
  0   0   0   0   0   0   0
  1   0   0  12  12  12  12
  2   0  10  12  22  22  22
  3   0  10  12  22  30  32
  4   0  10  15  25  30  37
```

**표 읽는 법** — 행 $i$ 는 물건 $1..i$ 까지만 고려한다는 뜻이고, 열 $w$ 는 현재 배낭 용량이다. 각 셀은 해당 조건에서 얻을 수 있는 최대 가치이다. 0행과 0열은 기저 조건(물건이 없거나 용량이 0)이므로 가치가 0 이다. 내부 셀은 "물건 $i$ 를 넣지 않은 경우 $K[i-1][w]$" 와 "넣는 경우 $v_i + K[i-1][w-w_i]$" 중 더 큰 값으로 정해진다. 예를 들어
$K[3][4] = \max(K[2][4] = 22, ; 20 + K[2][4-3] = 20 + 10 = 30) = 30$ 이다.

**정답** — $K[4][5] = 37$ 이다.

**역추적** — $K[i][w] \neq K[i-1][w]$ 이면 물건 $i$ 를 넣은 것이므로 선택 집합에 추가하고 $w$ 를 $w - w_i$ 로 줄인다.

**복잡도** — $O(nC)$ 이며, 이를 의사다항시간(Pseudopolynomial Time)이라 부른다. 숫자 값 $C$ 에 대해서는 다항이지만 비트 길이 $\log C$ 에 대해서는 지수이기 때문이다.

**공간 최적화** — 점화식이 직전 행 $i-1$ 만 참조하므로 1D 배열로 줄일 수 있다. 단, 이 경우 반드시 $w$ 를 오른쪽에서 왼쪽으로(역순으로) 갱신해야 같은 물건이 중복 사용되지 않는다.

**그리디가 실패하는 이유** — 가치/무게 비율이 큰 순서로 담으면 용량이 남았음에도 다음 물건이 들어가지 못하는 상황이 생긴다. DP 는 모든 부분해를 탐색하므로 이러한 경우도 올바르게 처리한다.

### 6.5.5 동전 거스름돈 (최소 동전 개수)

**문제 정의** — 동전 단위 $d_1, \ldots, d_k$ 와 금액 $n$ 이 주어졌을 때, $n$ 을 만드는 최소 동전 수를 구한다.

**부분 문제** — $C[j]$ 는 금액 $j$ 를 만드는 최소 동전 수이다.

**점화식** — $C[j] = \min_{d_i \leq j} C[j - d_i] + 1$ 이며, $C[0] = 0$ 이다.

**예시** — 동전 단위가 $1, 3, 4$ 이고 금액이 6일 때 DP 테이블은 다음과 같다.

| $j$ | **0** | **1** | **2** | **3** | **4** | **5** | **6** |
|---|---|---|---|---|---|---|---|
| $C[j]$ | 0 | 1 | 2 | 1 | 1 | 2 | **2** |

**그리디와의 비교** — 그리디는 4 + 1 + 1 = 3개를 선택하지만, DP 는 3 + 3 = 2개로 최적해를 구한다.

**복잡도** — $O(nk)$ 이다.

### 6.5.6 플로이드-워셜 — 모든 쌍 최단 경로

**문제 정의** — 가중 방향 그래프가 주어졌을 때, 모든 정점 쌍 사이의 최단 거리를 구한다.

**부분 문제** — $d[i, j, k]$ 는 $i$ 에서 $j$ 로 가는 경로 중 중간 정점으로 $1, \ldots, k$ 만 사용한 최단 거리이다.

**점화식** — $d[i, j, k] = \min(d[i, j, k-1], ; d[i, k, k-1] + d[k, j, k-1])$ 이다.

**구현** (공간 절약을 위해 3차원 배열 없이):

```
FloydWarshall(W, n):
    D = W                              # 인접 행렬 복사
    for k = 1 to n:                    # [주의] k 가 가장 바깥!
        for i = 1 to n:
            for j = 1 to n:
                D[i,j] = min(D[i,j], D[i,k] + D[k,j])
    return D
```

**설명** — $D$ 는 "중간 경유 정점 집합이 $1..k$ 일 때의 모든 쌍 최단 거리" 를 저장하는 2D 배열이다. $k$ 를 하나씩 늘리면서 "$k$ 를 경유하는 경로가 더 짧은가?" 를 모든 $(i, j)$ 에 대해 검사하고 갱신한다. 3차원 배열을 쓰지 않고 같은 2D 배열 $D$ 를 덮어써도 정답이 나오는 이유는, $k$ 번째 갱신 시 $D[i][k]$ 와 $D[k][j]$ 가 현재 반복에서 갱신되더라도 그 값이 동일하기 때문이다 (자기 자신을 경유하는 것은 의미가 없다).

**루프 순서** — $k$ 는 반드시 가장 바깥 루프여야 한다. $k$ 는 "허용되는 중간 정점 집합" 을 확장하는 역할을 하므로, 순서를 바꾸면 오답이 된다.

**복잡도** — 시간 $O(n^3)$, 공간 $O(n^2)$ 이다.

**음수 간선** — 음의 사이클만 없다면 음수 간선도 허용된다. 음의 사이클은 대각선 $D[i][i] < 0$ 으로 감지할 수 있다.

### 6.5.7 행렬 경로 (최대 합)

**부분 문제** — $c[i, j]$ 는 $(1, 1)$ 에서 $(i, j)$ 까지의 최대 누적 합이며, 오른쪽 또는 아래로만 이동할 수 있다.

**점화식** — $c[i, j] = m[i, j] + \max(c[i-1, j], c[i, j-1])$ 이다.

**복잡도** — 시간·공간 모두 $O(n^2)$ 이다.

## 6.6 DP 문제 총정리

| 문제 | 부분 문제 | 점화식 | 시간 | 공간 |
|---|---|---|---|---|
| 피보나치 | $f[i]$ | $f[i-1] + f[i-2]$ | $O(n)$ | $O(1)$ |
| 행렬 경로 | $c[i, j]$ | $m[i, j] + \max(c[i-1, j], c[i, j-1])$ | $O(n^2)$ | $O(n^2)$ |
| LCS | $c[i, j]$ | 일치/불일치 분기 | $O(mn)$ | $O(mn)$ |
| 편집 거리 | $E[i, j]$ | 삽입·삭제·교체 중 min | $O(mn)$ | $O(mn)$ |

| | | | | |
|---|---|---|---|---|
| 0/1 배낭 | $K[i, w]$ | 포함/제외 중 max | $O(nC)$ | $O(nC)$ |
| 동전 (최소) | $C[j]$ | $\min\,(C[j-d_i]+1)$ | $O(nk)$ | $O(n)$ |
| 플로이드-워셜 (Floyd-Warshall) | $d[i, j, k]$ | 중간 정점 $k$ 포함/미포함 | $O(n^3)$ | $O(n^2)$ |

---

## 6장 정리

**큰 그림** — DP 는 최적 부분 구조와 중복 부분 문제를 이용해 지수 시간 재귀를 다항 시간으로 줄이는 기법이다. 표준 6 단계 레시피는 부분 문제 정의, 점화식 도출, 기저 설정, 계산 순서 결정, 답 추출, 역추적이다. 7 대 고전 문제인 피보나치, 행렬 경로, LCS, 편집 거리, 0/1 배낭, 동전 거스름돈, 플로이드-워셜을 마스터해야 한다.

---

# 7장. 중간고사 복습 — 종합

## 7.1 학기 전체를 관통하는 흐름

```
문제 해결
    ↕
분할 정복 ↔ 재귀적 사고 ↔ 점화식
    ↕
패러다임 선택 (그리디 / DP / D&C)
```

모든 알고리즘은 "큰 문제 → 작은 문제 → 조합" 이라는 **재귀적 사고** 를 공유한다. 패러다임 간 차이는 부분 문제의 독립성 여부와 부분 문제 정의의 디테일에서 발생한다.

## 7.2 복잡도 치트시트

### 7.2.1 복잡도 계층

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n\log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

### 7.2.2 정렬 완전 표

| 알고리즘 | 최악 | 평균 | 최선 | 공간 | 안정 | 제자리 |
|---|---|---|---|---|---|---|
| 선택 | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n^2)$ | $O(1)$ | × | ✓ |
| 버블 | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n)$ | $O(1)$ | ✓ | ✓ |
| 삽입 | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n)$ | $O(1)$ | ✓ | ✓ |
| 병합 | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $O(n)$ | ✓ | × |
| 퀵 | $\Theta(n^2)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $O(\log n)$ | × | ✓ |
| 힙 | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $O(1)$ | × | ✓ |
| 계수 | $\Theta(n+k)$ | $\Theta(n+k)$ | $\Theta(n+k)$ | $O(k)$ | ✓ | × |
| 기수 (Radix) | $\Theta(d(n+k))$ | $\Theta(d(n+k))$ | $\Theta(d(n+k))$ | $O(n+k)$ | ✓ | × |

### 7.2.3 마스터 정리 빠른 참조

$T(n) = aT(n/b) + f(n)$, $h = n^{\log_b a}$:

| $f$ vs $h$ | 결과 |
|---|---|
| $f = O(h/n^{\varepsilon})$ | $\Theta(h)$ (Case 1) |
| $f = \Theta(h)$ | $\Theta(h\log n)$ (Case 2) |
| $f = \Omega(h \cdot n^{\varepsilon})$ + 정규성 | $\Theta(f)$ (Case 3) |

### 7.2.4 점화식 카탈로그

| 점화식 | 해 | 예시 |
|---|---|---|
| $T(n) = T(n/2) + 1$ | $\Theta(\log n)$ | 이진 탐색 |
| $T(n) = T(n-1) + 1$ | $\Theta(n)$ | 팩토리얼 |
| $T(n) = 2T(n/2) + n$ | $\Theta(n\log n)$ | 병합 정렬 |
| $T(n) = 2T(n/2) + 1$ | $\Theta(n)$ | — |
| $T(n) = T(n-1) + n$ | $\Theta(n^2)$ | 삽입 정렬(Insertion Sort), 퀵 최악 |
| $T(n) = 2T(n-1) + 1$ | $\Theta(2^n)$ | 하노이 탑 |
| | | |

| | | |
|---|---|---|
| $T(n) = T(n-1) + T(n-2) + 1$ | $\Theta(\phi^n)$ | 순진 재귀 피보나치 |

## 7.3 DP 점화식 모음

- **LCS**: 일치 시 $c[i-1, j-1] + 1$, 불일치 시 $\max(c[i-1, j], c[i, j-1])$
- **편집 거리**: 일치 시 $E[i-1, j-1]$, 불일치 시 $1 + \min$ (세 방향)
- **0/1 배낭**: $\max(K[i-1, w], v_i + K[i-1, w-w_i])$
- **행렬 경로**: $m[i, j] + \max(c[i-1, j], c[i, j-1])$
- **동전 거스름돈**: $1 + \min_{d_i \le j} C[j - d_i]$
- **플로이드-워셜**: $\min(d[i, j], d[i, k] + d[k, j])$ (k 바깥 루프)

## 7.4 자주 하는 실수와 예방

| 실수 | 예방 |
|---|---|
| O vs Ω vs Θ 혼동 | 정의 암기, 양방향 한계 구분 |
| 느슨한 Big-O | 항상 타이트하게 (필요시 Θ 사용) |
| 증명 없이 그리디 주장 | 반례 탐색 → 없으면 교환 논증 |
| DP 기저 잘못 | n=0,1,2 손으로 검증 |
| DP 테이블 off-by-one | 0-index vs 1-index 명시 |
| 분할 가능 vs 0/1 배낭 | 쪼갤 수 있음 = 그리디, 없음 = DP |
| 퀵 정렬 정렬된 입력 | 랜덤 또는 중앙값-3 피벗 |
| 플로이드-워셜 루프 순서 | **k 는 반드시 가장 바깥** |
| Dijkstra + 음수 간선 | 벨만-포드(Bellman-Ford)로 교체 |
| 귀납: $cn + 1 \le cn$? | 추측 강화: $cn - d$ |
| 병합 정렬 공간 누락 | $O(n)$ 보조 배열 명시 |

---

# 8장. 큰 그림 — 세 패러다임의 관계

## 8.1 세 패러다임 비교

| 측면 | 분할 정복 | 그리디 | DP |
|---|---|---|---|
| **부분 문제** | 독립적 (트리) | N/A | **중복** (DAG) |
| **접근** | 분할 → 정복 → 결합 | 지역 최적 선택 | 부분 문제 저장·재사용 |
| **방향** | 재귀 (탑다운) | 순차 | 상향식 또는 메모이제이션 |
| **최적 보장** | 문제 의존 | 그리디 선택 속성 필요 | ✓ (최적 부분 구조) |
| **시간** | $O(n\log n)$ 전형 | $O(n\log n)$ / $O(n)$ | $O(n^2)$ / $O(n^3)$ |
| **공간** | $O(\log n)$ 스택 | $O(1) \sim O(n)$ | $O(n) \sim O(n^2)$ |
| **대표** | 병합·퀵·이진 탐색 | Dijkstra·Huffman·MST | LCS·편집·배낭·FW |

## 8.2 결정 흐름도

```
최적값을 구하는 문제?
├── 아니오 → D&C (정렬·탐색·k번째 등)
└── 예
    ├── 안전한 그리디 선택 증명 가능?
    │   └── 예 → 그리디
    └── 아니오
        ├── 부분 문제 겹침?
        │   └── 예 → DP
        └── 아니오 → D&C
```

## 8.3 같은 문제, 다른 패러다임

| 문제 | 그리디 | DP | D&C |
|---|---|---|---|
| 동전 거스름돈 (표준) | ✓ | ✓ | — |
| 동전 거스름돈 (임의) | × | ✓ | — |
| 분할 가능 배낭(Fractional Knapsack) | ✓ | ✓ | — |
| 0/1 배낭 | × | ✓ | — |
| 최단 경로 (음 ≥ 0) | ✓ Dijkstra | ✓ | — |
| 최단 경로 (음수 허용) | — | ✓ 벨만-포드·FW | — |
| 정렬 | — | — | ✓ |

| | | | |
|---|---|---|---|
| k 번째 원소 | — | — | ✓ 선택 |
| MST | ✓ | — | — |
| LCS / 편집 거리 | × | ✓ | — |

## 8.4 핵심 연결 고리 일곱 가지

### 연결 1: 재귀적 사고의 통일성

유클리드 GCD (1주차), 병합 정렬 (4주차), DP (6주차) 모두 "문제를 더 작은 문제로 환원" 하는 구조이다. 재귀는 전 과목을 관통하는 통주저음이다.

### 연결 2: 분할 정복과 로그 복잡도

이진 탐색 (1주차), 가짜 동전 (1주차), 병합 정렬 (4주차) 모두 "절반씩 분할" 하는 구조를 가지며, $\log n$ 레벨 × 각 레벨 $O(n)$ 또는 $O(1)$ 로 총 복잡도가 결정된다.

### 연결 3: 그리디에서 DP 로의 전환

임의 단위의 동전 거스름돈과 0/1 배낭에서는 그리디가 실패하기 때문에 DP 로 전환해야 한다. 이 전환 과정은 **반례 인식에서 일반화된 해법으로의 이동** 이라는 교훈을 보여준다.

### 연결 4: 비교 하한과 선형 정렬의 위계

비교 기반 $\Omega(n\log n)$ 하한은 **비교 모델의 한계** 이다. 모델을 바꿔 자릿수나 값의 범위 정보를 활용하면 이 하한을 깨고 $O(n)$ 이 가능해지며, 이것이 계수 정렬과 기수 정렬의 정당성이다.

### 연결 5: Prim ↔ Dijkstra ↔ Huffman

세 알고리즘은 모두 **"최소값 추출 → 확정 → 이웃 갱신"** 이라는 공통 패턴을 공유한다. 즉 우선순위 큐 기반 그리디 프레임워크라는 같은 골격 위에 서 있다.

### 연결 6: 분할 정복의 한계에서 DP 로

독립적인 부분 문제는 분할 정복이 처리하고, 중복되는 부분 문제는 DP 가 처리한다. 피보나치의 순진 재귀가 $O(2^n)$ 인 것이 이 경계를 극명히 보여주는 예이다.

### 연결 7: 복잡도 분석이 모든 것을 연결

Big-O, 점화식, 마스터 정리 (2주차) 는 3주차부터 7주차까지 모든 알고리즘 분석의 도구이다. 복잡도 언어 없이는 "왜 이 알고리즘이 저것보다 낫다" 를 설명할 수 없다.

## 8.5 답안 작성 심화 팁

### 패턴 1 — "왜" 를 먼저

× "LCS 는 DP 로 푼다."

✓ "LCS 는 $X[i] = Y[j]$ 일 때와 아닐 때의 두 가지 선택이 있고, 같은 부분 문제 $(i', j')$ 가 여러 경로에서 반복 등장한다. 그래서 DP 로 풀어야 지수에서 $O(mn)$ 으로 축소된다."

### 패턴 2 — 트레이드오프

× "퀵 정렬은 빠르다."

✓ "퀵 정렬은 평균 $\Theta(n\log n)$ 으로 병합 정렬과 동일하지만, 최악 $\Theta(n^2)$ 위험이 있다. 이를 피하려면 랜덤 피벗을 사용. 제자리 정렬(In-place Sort)이라 메모리 효율이 좋아 실전에서 가장 많이 쓰인다."

### 패턴 3 — 개념 연결

× "Prim 은 MST 를 찾는다."

✓ "Prim 은 Dijkstra 와 구조적으로 동일하다 — 최소값 추출 + 이웃 갱신. 차이는 $D[v]$ 의 의미뿐 — Prim 에서는 '트리까지의 최소 간선', Dijkstra 에서는 '$s$ 에서의 최단 거리'."

### 패턴 4 — 한계 언급

× "Dijkstra 로 최단 경로를 구한다."

✓ "Dijkstra 는 **음이 아닌 간선** 에서만 작동한다. 음수 간선이 있으면 확정된 $D[u]$ 가 나중에 더 작아질 수 있어 그리디 불변식이 깨진다. 이 경우 벨만-포드($O(VE)$)를 써야 한다."

### 패턴 5 — 실무 예시

× "LCS 는 O(mn) 이다."

✓ "LCS 는 Git diff 의 기반이며, DNA 서열 정렬에도 쓰인다. 두 파일의 공통 줄을 찾아 변경·추가·삭제를 표시한다. $O(mn)$ 이 너무 크면 Myers 알고리즘 같은 최적화로 $O((m+n)D)$ ($D$ = 편집 거리) 달성."

---

# 9장. 시험 직전 체크리스트

## 9.0 주차별 핵심 치트시트

7주차 수업에서 공유된 "Exam Review Cheat Sheet" 를 확장한 것이며, 시험장에서 가장 빨리 찾아야 할 핵심 정보를 모아둔 것이다.

| 주차 | 주제 | 핵심 알고리즘 | 복잡도 |
|---|---|---|---|
| 2주차 | 복잡도 | Big-O, 루프 분석 | — |
| 3주차 | 정렬 | 병합 정렬, 퀵 정렬, 힙 정렬 | $O(n\log n)$ |
| 4주차 | 분할 정복 | 이진 탐색, 최근접 쌍, 선택 | $O(\log n)$, $O(n\log n)$, $O(n)$ |
| 5주차 | 그리디 | 활동 선택, Huffman, 분할 가능 배낭, MST, Dijkstra | $O(n\log n)$ |
| 6주차 | DP | 피보나치, LCS, 0/1 배낭, 편집 거리, 플로이드-워셜 | $O(n)$, $O(mn)$, $O(nW)$, $O(n^3)$ |

### 다섯 가지 시험 팁

```
1. 복잡도:  i *= 2 (log n) vs i += 1 (n) 패턴을 알아차려라
2. 정렬:    각 정렬의 시간·공간 복잡도 AND 안정성을 외워라
3. D&C:     점화식을 먼저 쓰고 → 마스터 정리 적용
4. 그리디:  그리디 선택 속성이 "왜" 성립하는지 설명하라
5. DP:      부분 문제 정의 → 점화식 → 기저 → 채우는 순서
```

---

## 9.1 자주 하는 실수

| 실수 | 회피 방법 |
|---|---|
| O, Ω, Θ 혼동 | O = 상한, Ω = 하한, Θ = 양방향 꼭 맞는 한계 |
| f(n) 이 다항식 아닌데 마스터 정리 적용 | 정규성 조건 확인. 안 되면 다른 방법 사용 |
| 증명 없이 그리디 최적이라 주장 | **항상 반례부터 찾아보기** |
| DP 점화식 기저 잘못 | 작은 입력(n=1, 2, 3)으로 추적 검증 |
| DP 테이블 off-by-one 에러 | 0-index 인지 1-index 인지 **명시** |

| | |
|---|---|
| 분할 가능 vs 0/1 배낭 혼동 | 분할 가능 = 그리디, 0/1 = DP |
| 퀵 정렬 최악 ($n^2$) 간과 | 랜덤 피벗·중앙값-3 사용해 회피 |

---

## 9.2 시험 문제 유형 예시

교수님이 7주차 에서 직접 보여주신 **대표 시험 유형 3 가지**.

### 예시 1 — 복잡도 분석 (마스터 정리 적용)

**문제** — 다음 점화식의 시간 복잡도(Time Complexity)를 마스터 정리로 구하라.

$$T(n) = 4T(n / 2) + n$$

**풀이**:

- $a = 4$, $b = 2$, $f(n) = n$
- $n^{\log_b a} = n^{\log_2 4} = n^2$
- $f(n) = n$ vs $n^2$ 비교
- $f(n) = O(n^{2-\varepsilon})$ ($\varepsilon = 1$) → **Case 1**
- **답** — $T(n) = \Theta(n^2)$ 이다.

### 예시 2 — 그리디 vs DP (동전 거스름돈)

**문제** — 동전 단위가 $1, 3, 4$ 이고 금액이 6 센트일 때, 최소 동전 수로 만드는 방법을 그리디와 DP 로 각각 구해 비교하라.

**그리디 풀이**:

- 4 (남음: 2), 1 (남음: 1), 1 (남음: 0)
- 결과: $4, 1, 1$ = **3 개**

**DP 풀이**:

| j | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| C(j) | 0 | 1 | 2 | 1 | 1 | 2 | **2** |

- $C(6) = 1 + \min(C(5), C(3), C(2)) = 1 + \min(2, 1, 2) = 2$
- 결과: $3, 3$ = **2 개**

**결론** — 그리디는 3 개, DP 는 2 개를 구하므로 이 동전 단위에서 그리디는 최적이 아니다.

### 예시 3 — DP

**문제** — $X = $ "ABCB" 와 $Y = $ "BDCAB" 의 LCS 를 DP 테이블로 구하라.

**테이블**:

| | "" | B | D | C | A | B |
|---|---|---|---|---|---|---|
| "" | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 0 | 0 | 0 | **1** | 1 |
| B | 0 | **1** | 1 | 1 | 1 | **2** |
| C | 0 | 1 | 1 | **2** | 2 | 2 |
| B | 0 | 1 | 1 | 2 | 2 | **3** |

**답** — LCS 길이는 3 이며, 하나의 LCS 는 "BCB" 이다.

**역추적** — 문자가 일치하면 대각선으로 이동해 해당 문자를 포함시키고, 일치하지 않으면 두 이웃 중 값이 큰 쪽으로 이동한다.

---

## 9.3 패러다임 식별 연습 10문제

7주차 Lab Exercise 2 에서 제시된 문제들로, 시험에서 "이 문제를 어떤 패러다임으로 풀 것인가" 를 판단하는 능력을 훈련하기 위한 것이다.

### 문제 1-5

| # | 문제 | 패러다임 | 이유 |
|---|---|---|---|
| 1 | Two Sum (n ≤ 100) | **완전 탐색** | $n^2 = 10{,}000$ — $O(n^2)$ 수용 가능 |
| 2 | 최대 부분 배열 합 | **DP** (Kadane) 또는 D&C | `dp[i] = max(a[i], dp[i-1]+a[i])` |
| 3 | 최근접 점 쌍(Closest Pair of Points) ($n \leq 10^5$) | **분할 정복** | x 분할, 두 반 각각 재귀, 스트립 검사. $O(n\log n)$ |
| 4 | 동전 거스름돈 {1, 5, 7, 10} | **DP** | 비표준 단위; 그리디 실패 |

| | | | |
|---|---|---|---|
| 5 | 활동 선택 | **그리디** | 종료 시간 오름차순 정렬, EFT 최적성 증명됨 |

**문제 4**:

```
동전 {1, 5, 7, 10}, 금액 14

그리디: 10 + 1 + 1 + 1 + 1 = 5 개
DP:     7 + 7              = 2 개  ← 최적!
```

## 문제 6-10

| # | 문제 | 패러다임 | 이유 |
|---|---|---|---|
| 6 | 문자열 순열 부분 (Permutation in String) | **완전 탐색** (슬라이딩 윈도우(Sliding Window)) | 길이 `len(s1)` 의 모든 윈도우 검사 |
| 7 | 최장 증가 부분 수열 (LIS) | **DP** | `dp[i]` = 인덱스 i 에서 끝나는 LIS 길이 |
| 8 | 큰 수 곱셈 | **분할 정복** (Karatsuba) | 자릿수 분할, 3 곱셈. $O(n^{1.585})$ |
| 9 | 최소 회의실 개수 | **그리디** | 이벤트 스윕(Sweep): 시작 +1, 끝 -1, 최댓값 추적 |
| 10 | 편집 거리 | **DP** | LCS 와 유사 2D 테이블. `dp[i][j]` = 접두사 최소 연산 |

## 패러다임 결정 플로우차트

```
                    문제 읽기
                       |
                       ▼
              제약이 작은가 (n ≤ 수백)?
                    /     \
                 예         아니오
                  |          |
                  ▼          ▼
            완전 탐색    독립 부분 문제로 분할 가능?
                            /          \
                         예             아니오
                          |              |
                          ▼              ▼
                     분할 정복     지역 최적이 전역 최적?
                                     /         \
                                  예            아니오
                                   |             |
                                   ▼             ▼
                                 그리디       중복 부분 문제 +
                                              최적 부분 구조?
                                                  |
                                                  ▼
                                                 DP
```

**경험칙** — 먼저 그리디를 시도하고, 반례가 발견되면 DP 로 전환한다.

---

## 9.4 복습 문제 다섯 가지

7주차 Lab Exercise 3 에 제시된 문제들로, 주차별 핵심 개념을 복습하기 위한 것이다.

### 복습 1 — 복잡도 분석

**문제** — 다음 함수의 시간 복잡도와 `mystery(16)` 결과는?

```python
def mystery(n):
    count = 0
    i = 1
    while i < n:          # 바깥 루프
        j = 0
        while j < n:      # 안쪽 루프
            count += 1
            j += 1
        i *= 2             # 핵심: i 가 2 배씩 증가
    return count
```

**풀이**:

```
바깥 루프: i = 1, 2, 4, 8, ..., i ≥ n 까지
            i 가 2 배 증가 → O(log n) 회
안쪽 루프: j = 0, 1, ..., n−1
            O(n) 회 per 바깥 반복
합계: O(n) × O(log n) = O(n log n)
```

`mystery(16)` **추적**:

```
i = 1:   j 가 0..15  → 16 회
i = 2:   j 가 0..15  → 16 회
i = 4:   j 가 0..15  → 16 회
i = 8:   j 가 0..15  → 16 회
i = 16:  while 조건 실패, 종료

count = 16 × 4 = 64
```

**답** — (a) $O(n\log n)$, (b) 64 이다.

**핵심** — `i *= 2` 패턴은 $\log n$ 번, `i += 1` 패턴은 $n$ 번 반복되며, 이는 시험에 빈번히 출제되는 패턴이다.

## 복습 2 — 병합 정렬

**문제** — 병합 정렬을 구현하고 병합(merge) 연산 횟수를 세시오.

```python
def merge_sort(arr):
    """(정렬된 배열, 병합 횟수) 반환."""
    if len(arr) <= 1:
        return arr[:], 0

    mid = len(arr) // 2
```

main 26Spring_Algorithms / Preparation / Mid_Total.ko.md ↑ Top

Preview Code Blame 2089 lines (1528 loc) · 16 B Raw

```python
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:      # <= (안정성)
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged, lc + rc + 1
```

**설명** — 중간 지점으로 배열을 두 부분으로 쪼개 재귀 호출. 반환되는 두 정렬된 부분 배열을 **투 포인터**로 병합 — `left[i]` 와 `right[j]` 중 작은 쪽을 결과에 추가하고 포인터 전진. 남은 원소는 `extend` 로 끝에 붙임. 병합 횟수는 좌·우의 병합 수 합에 **이번 병합 1회** 를 더해 집계. 안정성을 위해 `<=` 를 씀(같은 값일 때 왼쪽을 먼저).

**예시** — 입력 `[38, 27, 43, 3, 9, 82, 10]` 은 `[3, 9, 10, 27, 38, 43, 82]` 으로 정렬되며, 병합 횟수는 6 회로 재귀 트리의 내부 노드 수 $n-1$ 과 같다.

## 복습 3 — 분할 정복으로 최댓값

**문제** — 분할 정복으로 배열 최댓값을 찾고 비교 횟수를 세시오.

```python
def find_max_dc(arr):
    """(최댓값, 비교 횟수) 반환."""
    comparisons = [0]

    def helper(a, lo, hi):
        if lo == hi:
            return a[lo]
        mid = (lo + hi) // 2
        left_max = helper(a, lo, mid)
        right_max = helper(a, mid + 1, hi)
        comparisons[0] += 1
        return left_max if left_max >= right_max else right_max

    return helper(arr, 0, len(arr) - 1), comparisons[0]
```

**설명** — 배열을 반으로 나눠 왼쪽·오른쪽 각각의 최댓값을 재귀로 구한 뒤 **1 회 비교** 로 둘 중 큰 값을 반환. 기저는 원소 1개( `lo == hi` ). 점화식 $T(n) = 2T(n/2) + 1$ 은 마스터 정리 Case 1 (f(n)=1, $n^{\log_2 2} = n$) 으로 $\Theta(n)$. 비교 횟수는 재귀 트리의 내부 노드 수 = $n - 1$ 로 정확히 최소.

**예시** — `[3, 7, 2, 9, 1, 8, 4, 6, 5]` 의 최댓값은 9 이며, 비교는 총 8 회로 $n - 1$ 과 같아 최적이다.

**점화식** — $T(n) = 2T(n/2) + 1$ 이며 해는 $O(n)$ 이다.

**시사점** — 비교 횟수 $n - 1$ 은 순차 탐색과 동일하다. 분할 정복이라고 해서 반드시 더 빠른 것은 아니며, 최댓값 찾기는 본질적으로 $\Theta(n)$ 이다.

## 복습 4 — 활동 선택

**문제** — (시작, 끝) 강의 목록에서 한 강의실로 수용 가능한 **최대 개수** 선택.

```python
def activity_selection(lectures):
    """(개수, 선택된 인덱스 리스트) 반환."""
    indexed = [(end, start, i) for i, (start, end) in enumerate(lect
    indexed.sort()                    # 종료 시간 기준 정렬 (EFT)

    selected = []
    last_end = -1
    for end, start, idx in indexed:
        if start >= last_end:
            selected.append(idx)
            last_end = end
    return len(selected), selected
```

**설명** — 각 강의를 `(end, start, 원본인덱스)` 튜플로 변환해 **종료 시간으로 정렬**(파이썬 튜플 정렬은 첫 원소 기준). 순회하면서 현재 강의의 시작이 지난 `last_end` 이상이면 선택하고 `last_end` 를 현재 종료로 갱신. "가장 일찍 끝나는 강의부터" 선택하는 그리디가 최적인 이유는, 그래야 이후 강의들을 위한 시간이 최대한 남기 때문(교환 논증).

**예시** — 강의 목록이 `[(1,3),(2,5),(3,6),(5,7),(6,8),(8,10),(9,11)]` 일 때, 선택되는 강의는 (1,3) → (5,7) → (8,10) 으로 총 3 개이다.

**핵심** — EFT (Earliest Finish Time, 가장 빨리 끝나는) 기준의 그리디이며, 정렬에 지배되어 시간 복잡도는 $O(n\log n)$ 이다.

## 복습 5 — 계단 오르기 DP

**문제** — $n$ 개 계단, 각 계단 비용 `cost[i]`. 1 칸 또는 2 칸씩 오르고 계단 0 또는 1 에서 출발. 꼭대기까지 **최소 비용**.

```python
def min_cost_climbing(cost):
    n = len(cost)
    if n <= 1:
        return cost[0] if n == 1 else 0

    prev2 = cost[0]        # dp[0]
    prev1 = cost[1]        # dp[1]

    for i in range(2, n):
        current = cost[i] + min(prev1, prev2)
        prev2, prev1 = prev1, current

    return min(prev1, prev2)    # 마지막 또는 마지막 직전에서 꼭대기로
```

**설명** — $dp[i]$ = 계단 $i$ 에 도착한 누적 최소 비용. 점화식 $dp[i] = \text{cost}[i] + \min(dp[i-1], dp[i-2])$ — 이전 또는 두 칸 전에서 올라왔는지 중 저렴한 쪽을 선택. 꼭대기는 마지막 계단 $n-1$ 또는 그 직전 $n-2$ 에서 한 걸음 더 가면 도달하므로 마지막에 $\min(dp[n-1], dp[n-2])$. **공간 $O(1)$ 최적화**: 점화식이 바로 이전 두 값만 필요하므로 배열 대신 두 변수( `prev1` , `prev2` )만 유지.

**예시**

- `[10, 15, 20]` 의 최소 비용은 15 이다.
- `[1, 100, 1, 1, 1, 100, 1, 1, 100, 1]` 의 최소 비용은 6 이다.

**점화식** — `dp[i] = cost[i] + min(dp[i-1], dp[i-2])` 이다.

**복잡도** — 시간 $O(n)$, 공간 $O(1)$ 이다 (두 변수만 유지하면 충분하다).

**일반화** — 2D DP 에서도 점화식이 직전 행 또는 직전 열만을 참조한다면 1D 배열로 공간을 축소할 수 있다.

---

## 9.6 자가 점검 30문제

### 1주차 기초

1. 알고리즘의 다섯 가지 속성은?
2. 유클리드 GCD 알고리즘의 정확성 근거는?
3. 이진 탐색에서 `left <= right` 의 등호가 필요한 이유?
4. 독이 든 포도주(Poisoned Wine) 문제에서 $\log_2 n$ 명이 충분한 이유?

### 2주차 복잡도

5. Big-O, Ω, Θ 의 차이?
6. RAM 모델이란?
7. 마스터 정리 Case 1, 2, 3 각각의 조건과 결과?
8. $T(n) = 2T(n/2) + n$ 을 귀납법으로 증명?

### 3주차 자료구조·정렬

9. 배열 vs 연결 리스트 연산별 복잡도?
10. 스택·큐의 LIFO vs FIFO 실전 용도 각 3 개?
11. 힙의 배열 표현에서 부모·자식 인덱스?
12. 삽입 정렬의 최선 $\Theta(n)$ 조건?
13. 퀵 정렬이 $\Theta(n^2)$ 로 가는 최악 시나리오?
14. buildHeap 이 $O(n)$ 인 이유?
15. 비교 기반 정렬의 하한 $\Omega(n\log n)$ 증명?
16. 계수 정렬에서 역순 순회가 필요한 이유?
17. 기수 정렬이 LSD 순서를 쓰는 이유?

### 4주차 분할 정복

18. 분할 정복 3 단계?
19. 병합 정렬이 최악도 $\Theta(n\log n)$ 인 이유?
20. 랜덤 선택의 평균 $\Theta(n)$ 증명 개요?
21. 중앙값의 중앙값에서 왜 5 개씩 묶는가?

22. 최근접 쌍의 스트립 검사에서 7 개 이웃만 충분한 이유?

### 5주차 그리디

23. 그리디 선택 속성과 최적 부분 구조의 차이?
24. 활동 선택(EFT) vs 작업 스케줄링(EST) 차이?
25. Huffman 코딩의 최적성 증명 개요?
26. Kruskal vs Prim 의 자료구조 차이?
27. Dijkstra 가 음수 간선에서 실패하는 이유?

### 6주차 DP

28. DP 의 두 필수 조건?
29. 메모이제이션 vs 테이블화 차이?
30. 0/1 배낭에서 그리디 실패, DP 성공의 이유?

## 9.7 계산 연습

### 마스터 정리

$T(n) = 7T(n/2) + n^2$ 의 해는 다음과 같이 구한다.

- $a = 7$, $b = 2$ 이므로 $n^{\log_2 7} \approx n^{2.81}$ 이다.
- $f(n) = n^2 = O(n^{2.81-\varepsilon})$ 이므로 Case 1 에 해당한다.
- 따라서 $T(n) = \Theta(n^{\log_2 7})$ 이다.

### LCS

$X = $ "ABCB", $Y = $ "BDCAB" 의 LCS 는 테이블을 채워 구할 수 있으며, $c[4][5] = 3$ 이고 LCS 는 "BCB" 이다.

### 0/1 배낭

물건이 $(w, v) = (2, 12), (1, 10), (3, 20), (2, 15)$ 이고 용량이 $C = 5$ 일 때 최대 가치를 구하면, 테이블의 $K[4][5] = 37$ 이며 물건 4, 2, 1 이 선택되어 총 무게 5 kg 이 된다.

### Big-O 증명

$5n^3 + 2n^2 + 7 = O(n^3)$ 임을 다음과 같이 증명한다.

- $c = 14$, $n_0 = 1$ 로 설정한다.

- 모든 $n \geq 1$ 에 대해 $5n^3 + 2n^2 + 7 \leq 5n^3 + 2n^3 + 7n^3 = 14n^3$ 이 성립한다.

## 9.8 의사코드 체크

### 이진 탐색

```
left, right = 0, n-1
while left <= right:
    mid = (left + right) // 2
    if A[mid] == key: return mid
    elif A[mid] < key: left = mid + 1
    else: right = mid - 1
return -1
```

### 병합 정렬

```
mergeSort(A, p, r):
    if p < r:
        q = (p+r)//2
        mergeSort(A, p, q)
        mergeSort(A, q+1, r)
        merge(A, p, q, r)
```

### LCS 채우기

```
for i = 1 to m:
    for j = 1 to n:
        if X[i] == Y[j]: c[i][j] = c[i-1][j-1] + 1
        else: c[i][j] = max(c[i-1][j], c[i][j-1])
```

### 0/1 배낭 채우기

```
for i = 1 to n:
    for w = 0 to C:
        if w[i] > w: K[i][w] = K[i-1][w]
        else: K[i][w] = max(K[i-1][w], v[i] + K[i-1][w-w[i]])
```

### Dijkstra

```
D[s] = 0; D[v] = inf for v ≠ s; S = {}
while S ≠ V:
    u = argmin D[v] for v ∉ S
    S = S ∪ {u}
    for v in neighbors(u):
        if v ∉ S and D[u] + w(u,v) < D[v]:
            D[v] = D[u] + w(u,v)
```

## 9.9 시험 당일 전략

### 시간 배분 (1시간 기준)

- 0~3 분: 전체 훑기
- 3~50 분: 답안 작성 (계산 먼저, 서술 나중)
- 50~55 분: 검토 (특히 fork/기저 조건/경계)
- 55~60 분: 여유

### 답안 작성 6요소 템플릿

```
[정의 1 문장] + [왜 필요한가 1 문장]
+ [메커니즘 2~4 문장]
+ [구체 예시/의사코드/테이블]
+ [한계·트레이드오프]
+ [다른 개념과 연결]
```

### 체크 포인트 — 자주 놓치는 것

- [ ] Big-O 증명 시 $c$ 와 $n_0$ 명시
- [ ] DP 기저 조건 명시 (0 행·0 열)
- [ ] Dijkstra 는 음수 간선 X
- [ ] 퀵 정렬 최악 = 정렬된 입력 + 마지막 피벗
- [ ] 비교 기반 하한 $\Omega(n\log n)$
- [ ] 플로이드-워셜 k 가 가장 바깥
- [ ] 역추적 시 화살표 방향 주의

---

## 맺음말

이 한 권으로 1주차~7주차 을 처음부터 이해할 수 있도록 **왜·메커니즘·예시·연결** 을 모두 담았습니다. 공부하시다가 "왜 이 개념이 여기 나오지?" 하는 의문이 들면 **8장 큰 그림**을 다시 읽으시면 구조가 잡힙니다.

"깊은 이해 답안" 의 핵심은 단순 암기가 아니라 **"이 개념은 왜 존재하고, 다른 어떤 개념과 맞물려 있으며, 어떤 트레이드오프를 갖는가"** 를 체화하는 것입니다.

**좋은 결과 있으시길 바랍니다.**

---

**Last Updated**: 2026-04-23